Modul 9

Objektrelationale Konzepte

Josef Altmann



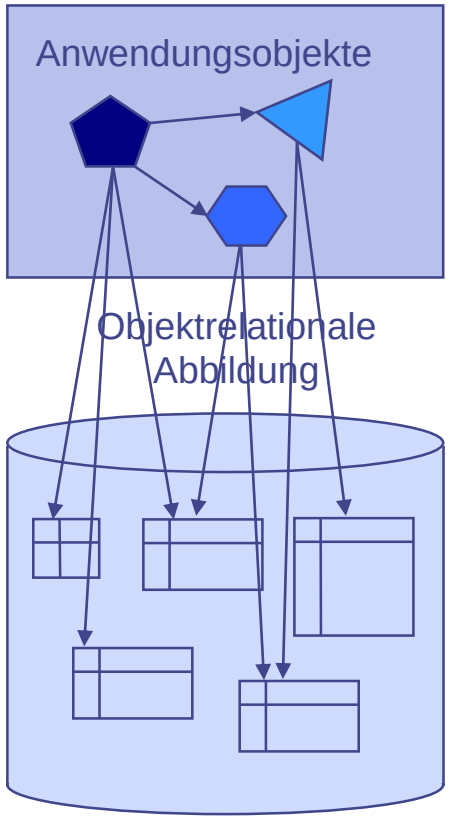
Der vorliegende Foliensatz basiert auf:

Kemper A., Eickler A.: Datenbanksysteme. Eine Einführung, 10. Auflage, Oldenbourg Verlag, München, 2015
Saake, G., Sattler, K.-U., Heuer, A.: Datenbanken: Konzepte und Sprachen, MITP Verlag, 6. Auflage, 2018
Faeskorn-Woyke, H., Bertelsmeier, B., Riemer, P., Bauer, E.: Datenbanksysteme, Pearson, 2007
Türker, C., Saake, G.: Objektrelationale Datenbanken, dpunkt, 2005
Türker, C.: SQL:1999 & SQL:2003: Objektrelationales SQL, dpunkt, 2003
Hohenstein, U., Pleßer, V.: Oracle 9i – Effiziente Anwendungsentwicklung mit objektrelationalen Konzepten, dpunkt, 2002

Inhalt

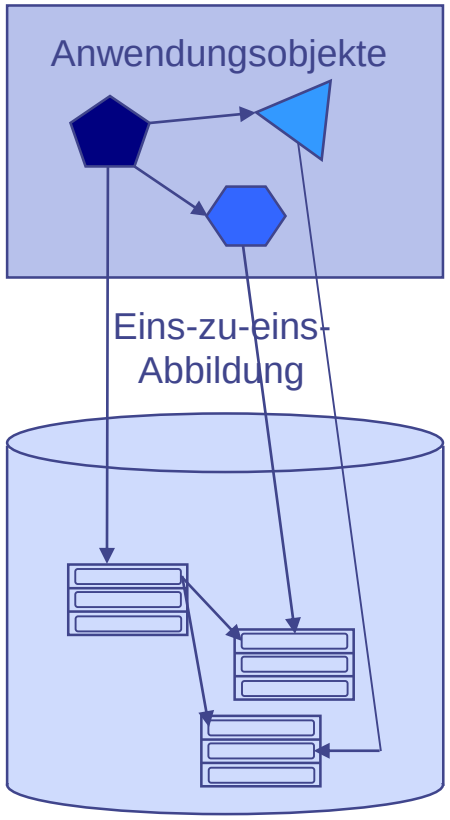
- Einführung und Motivation
- Objektrelationale Konzepte
- Objektrelationale Konzepte in Oracle
  - Einführung
  - Benutzerdefinierte Typen, Tabellen und Vererbung
  - Beziehungen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Relationale Darstellung
in flachen Tabellen



- Anwendungsentwickler steht vor folgenden Herausforderungen:
  - komplexe Objekte müssen zerlegt und auf viele Tabellen verteilt werden
  - zur Rekonstruktion der Objekte sind zeitaufwendige Verbundoperationen notwendig
  - durch die Abbildung zwischen den Datenmodellen (OO – Relational) entstehen Informationsverluste
  - die den Objekten zugeordneten Methoden stehen nicht zentral in der Datenbank zur Verfügung
- Unterstützung beispielsweise durch Middleware-Ebene oder Object Wrapper
  - EJB (Enterprise Java Beans)
  - JDO (Java Data Objects)
  - JPA (Java Persistence API)
  - Entity Framework für .NET
  - (N)Hibernate (OR-Mapping Werkzeug)
  - …

Objektrelationale Darstellung
in Objektabellen



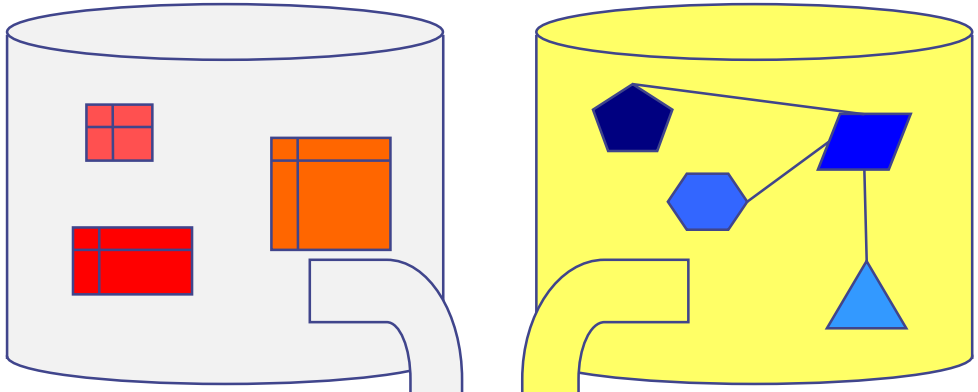
Objektrelationale DB

- Anwendungsentwickler hat folgende Vorteile:
  - das gleiche Datenmodell in Datenbank und Anwendungen erübrigt Abbildungen
  - die Objektmethoden sind in der Datenbank gespeichert und überall ausführbar (auf Server und Clients)
  - die ausgereiften Implementierungen und die bewährten Vorzüge der relationalen Datenbanksysteme bleiben erhalten
  - die systemübergreifende Norm SQL:2003 garantiert Herstellerunabhängigkeit (☹)
  - es sind keine neuen Systeme notwendig, die bisherigen Anwendungen laufen unverändert weiter
- Unterstützung beispielsweise durch
  - Oracle
  - IBM DB2
  - PostgreSQL
  - MS SQL Server (☹)
  - ...

Objektrelationale Konzepte

Synthese OO + RDB = ORDB

Relationales Datenmodell bzw. RDBS



Objektorientiertes Datenmodell

Vorteile:

- Reife (Speicherverwaltung, Optimierung, Recovery, Zugriffskontrolle, Transkationen)
- Stored Procedures in DB gespeichert und auf Server und Client ausführbar
- Standardkonformität (SQL)
- Verbreitung, etc.

Vorteile:

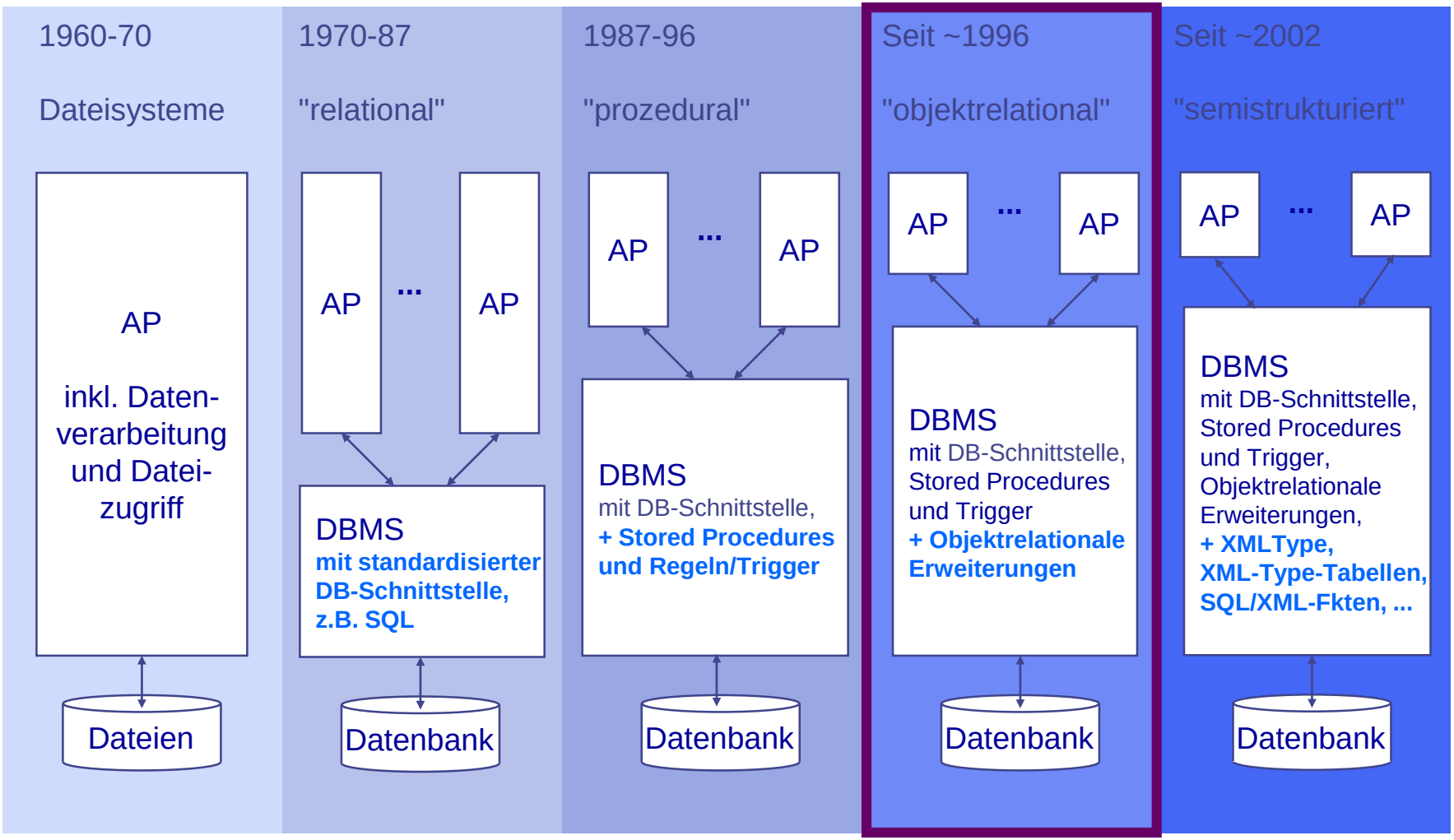
- Semantisch reiches Datenmodell
- Erweiterbarkeit des Typsystems
- Methoden, etc.

ORDBS

Vorteile:

- SQL-Standard als Voraussetzung für Herstellerunabhängigkeit
- Bisherige Anwendungen/Daten bleiben unverändert

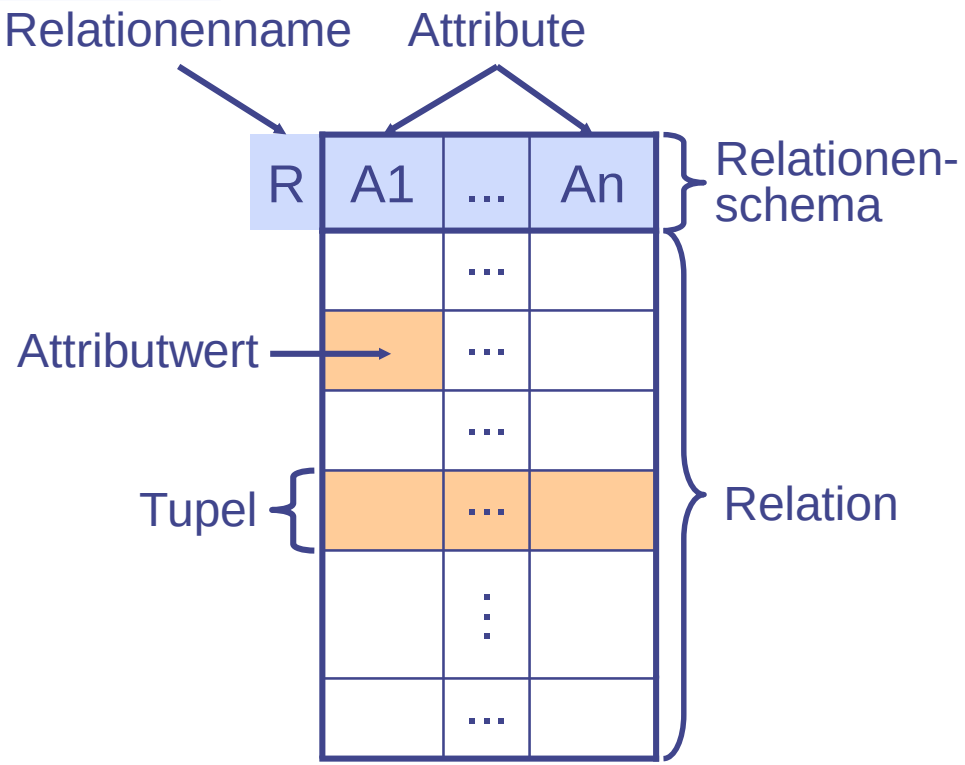
Evolution der Datenbanksysteme

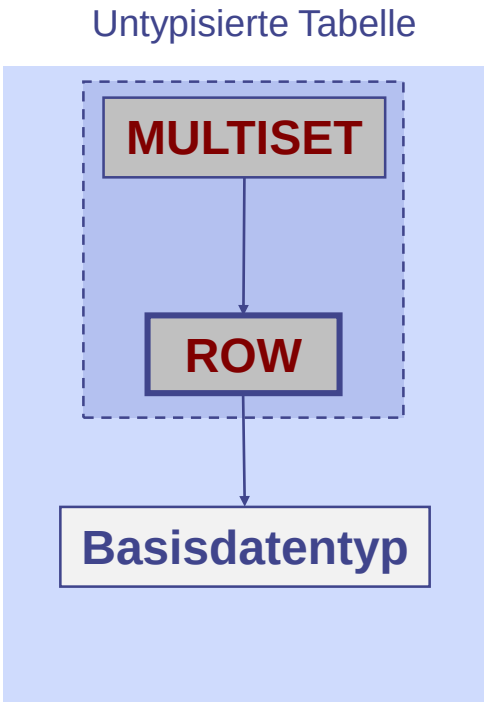


Inhalt

- Einführung und Motivation ✓
- Objektrelationale Konzepte
- Objektrelationale Konzepte in Oracle
  - Einführung
  - Benutzerdefinierte Typen, Tabellen und Vererbung
  - Beziehungen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Typsystem von SQL:92
Relationenmodell





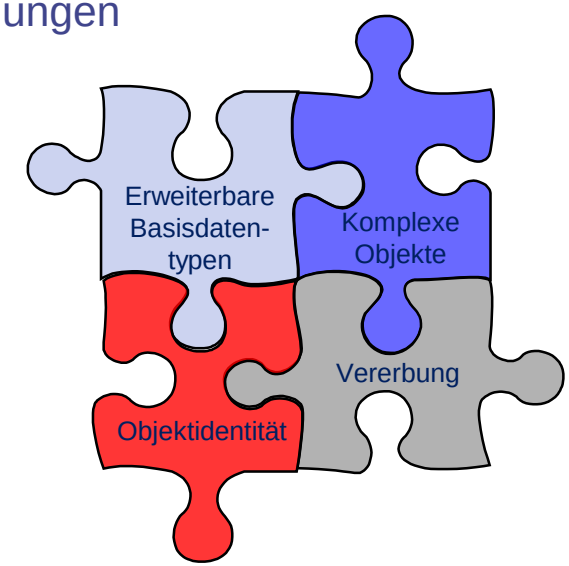
Einstiegspunkt in die Datenbank:
Untypisierte Tabelle: `MULTISET(ROW( TYPE(Sn), ...))`

```
ROW(Strasse VARCHAR(30), Nr DECIMAL(4), PLZ DECIMAL(5),...)
```

Objektrelationale Konzepte

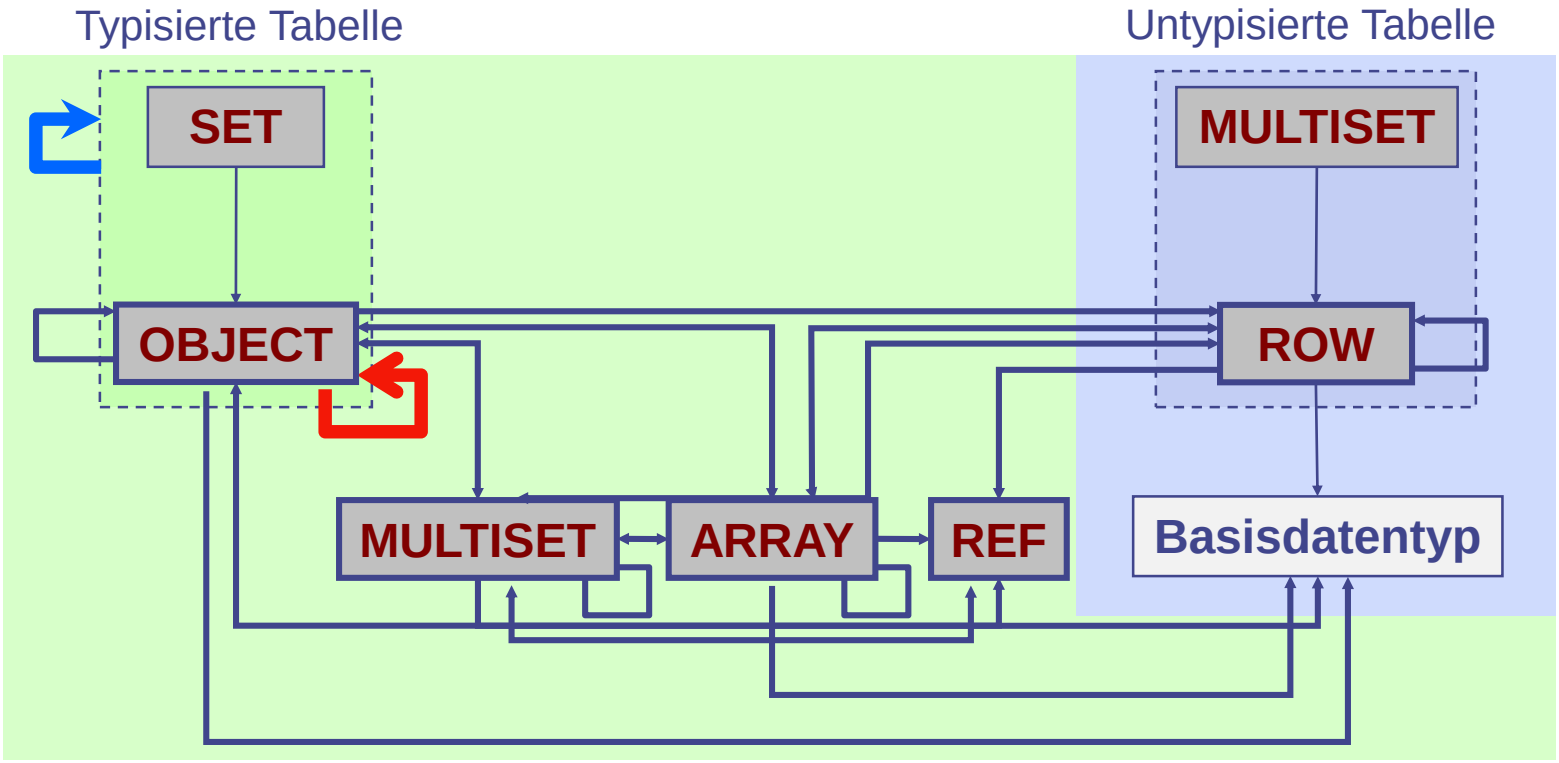
Erweiterung (SQL:1999 & SQL:2003 & SQL:2008) um …

- Benutzerdefinierte und komplexe Objekte/Datentypen
  - navigierende Anfragen
- Objektidentität (OID) und Referenzen
  - Objekte besitzen eindeutigen OID
  - Identische Objekte haben die gleiche OID; gleiche Objekte denselben Objektwert
  - OID zur Darstellung von Objektreferenzen/-beziehungen
- Vererbung
  - Subtypen und Subtabellen
- Polymorphie
  - Überschreiben und spätes Binden
- Kollektionstypen
  - Mengen, Multimengen, Listen, Arrays, ...
- Basisdatentypen
  - `BOOLEAN`, `BLOB`, `CLOB`, `XML`, ...
- etc.



Objektrelationale Konzepte

Typsystem von SQL:2008



Einstiegspunkte in die Datenbank:
Typisierte Tabelle: `SET(OBJECT(...))`
Untypisierte Tabelle: `MULTISET(ROW(...))`

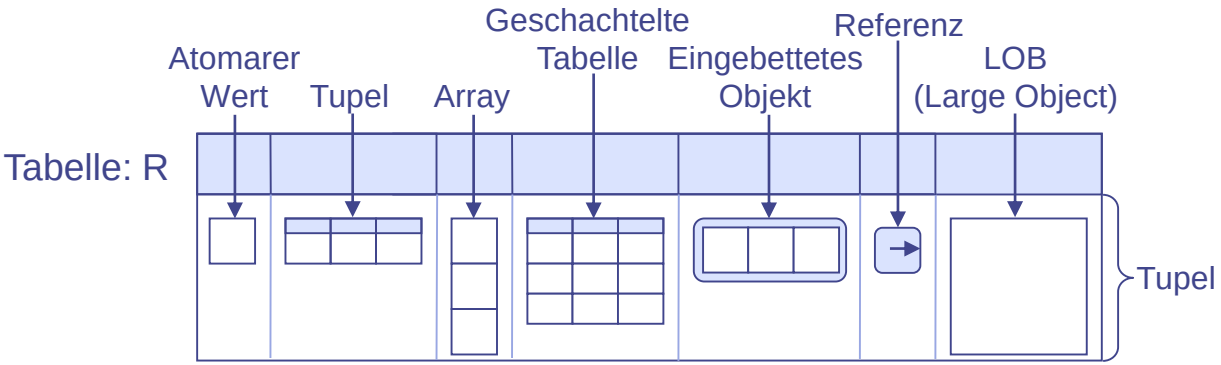
Subtypbbildung

Subtabellenbildung

Einstiegspunkt

Objektrelationale Konzepte
Tupeltabellen

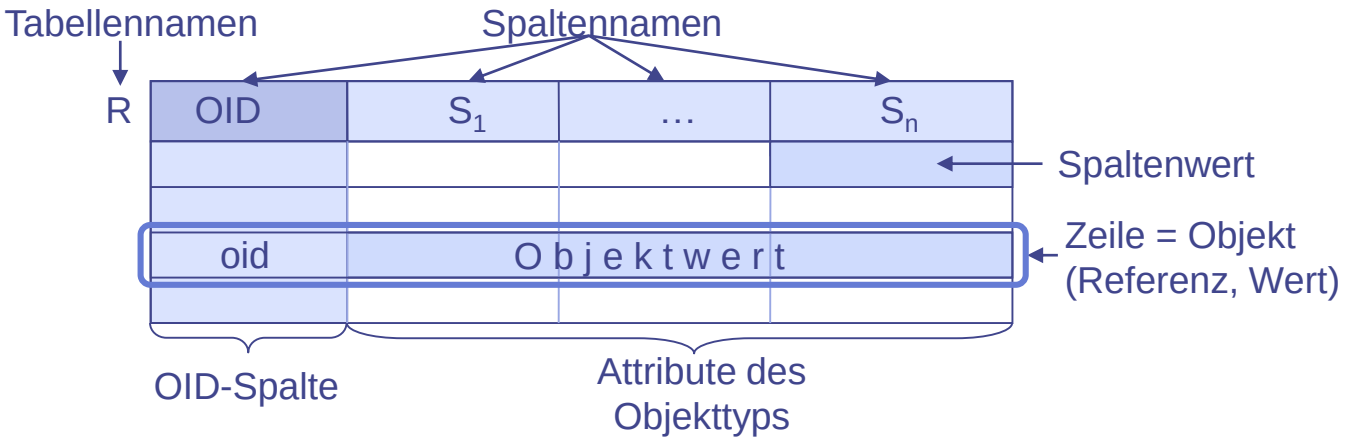
- Bis SQL-92:
  - nur Tupeltabellen; Attribute können nur Basisdatentypen sein
- Erweiterungen in SQL:1999 & SQL:2003 & SQL:2008
  - Attribute können array-, multimengen-, tupel-, objekt- und referenzwertig sein



- Typ einer Spalte in einer untypisierten Tupeltabelle ist mit der objektrelationalen Erweiterung des Typsystems nicht mehr nur auf Basisdatentypen beschränkt.

Objektrelationale Konzepte
Typisierte Tabellen

- Typisierte Tabelle basiert auf strukturierten Objekttyp
- Zeilen repräsentieren Objekte
- Zeilen haben eine OID-Spalte und sind referenzierbar
- Methoden können auf Zeilen (Objekte) aufgerufen werden



⇨ Typisierte Tabellen (~ Objekttabellen) entsprechen in etwa dem Klassenkonzept

Objektrelationale Konzepte

Tupeltabelle vs. Objekttabelle

Mitarbeiter (Tupeltabelle mit nicht-atomaren Attributen)

Name	Adresse <Straße,PLZ,Ort>	Hobbies	Bewerbung	Dienstwagen	Foto
VARCHAR	ROW(VARCHAR, INT, VARCHAR)	SET(CHAR)	OBJECT(...)	REF(Auto)	BLOB
'Altmann'	ROW('Irgendwogasse',4040, 'Linz')	SET('Kino', 'Sport')	OBJECT(...)	X'123534243646'	X'...'
…					
atomare Spalte	tupelwertige Spalte	mengenwertige Spalte	objektwertige Spalte	referenzwertige Spalte	LOB Spalte

Dienstwagen (Objekttabelle)

Objekttyp

OID	Hersteller	Farbe	PS	Motor
	VARCHAR	VARCHAR	INTEGER	OBJECT(...)
X'123534243646'	'Audi'	'Schwarz'	160	OBJECT(...)
…				

OID-Spalte

Objektwerte

zustandsunabhängige, unveränderbare Identifikation von Datenbankobjekten

- Objekttabelle basiert auf einem Objekttyp
- Zeilen sind Objekte und mittels Referenzspalten referenzierbar

Details siehe Anhang I

SQL:2008

Neue Datentypen

- `BLOB, CLOB`: Datentypen für große Binär-/Zeichenkettenobjekte (`binary/character large objects`)
- `BOOLEAN`: boolesche Werte `TRUE, FALSE, UNKNOWN`
- `ARRAY`: Kollektionstyp für mehrwertige Attribute (Ordnung!)
- `MULTISET`: Kollektionstyp für ungeordnete Multimengen (mit Duplikaten)
- `REF`-Typen: zur eindeutigen Identifikation von Tupeln (OID) sowie zur Navigation über Pfadausdrücke
- `ROW`-Typen: Typkonstruktor für strukturierte Attribute
- benutzerdefinierte Datentypen: Konstruktion neuer Typen
- ...

SQL:2008
Status

- Mangelhafte Umsetzung des SQL-Standards (rudimentäre Implementierungen großer DBS-Hersteller)
- ➢ Gefahr, sich an die Spezialitäten eines DBS-Herstellers zu binden (Portabilität)
- ➢ Jedoch Marktführer (Oracle, DB2, ...) setzen auf diese Technologie
- ➢ Daher Vorstellung der objektrelationalen Konzepten am Beispiel von Oracle 12c

Typkonstruktoren	Oracle	DB2	Informix	SQL Server
ROW	-		✓	
ARRAY	✓			
MULTISET	✓		✓	
SET	-		✓	
LIST	-		✓	
OBJECT	✓	✓		
OID/REF	✓	✓		
Typisierte Tabellen	✓	✓	✓	
Typisierte Sichten	✓	✓	eingeschränkt	
Vererbung	✓	✓	✓	

Inhalt

- Einführung und Motivation ✓
- Objektrelationale Konzepte ✓
- Objektrelationale Datenbanken in Oracle
  - Einführung
  - Benutzerdefinierte Typen, Tabellen und Vererbung
  - Beziehungen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Einführung 1/4
OO Konzepte im Oracle Datenmodell

- Benutzerdefinierte Datentypen
  - Objekttyp: definiert Datenstruktur und Methoden von Objekten
  - Referenztyp: realisiert Referenzen auf OID von Objekten
  - Tabellentyp und Array-Typ: realisieren mehrwertige Attribute

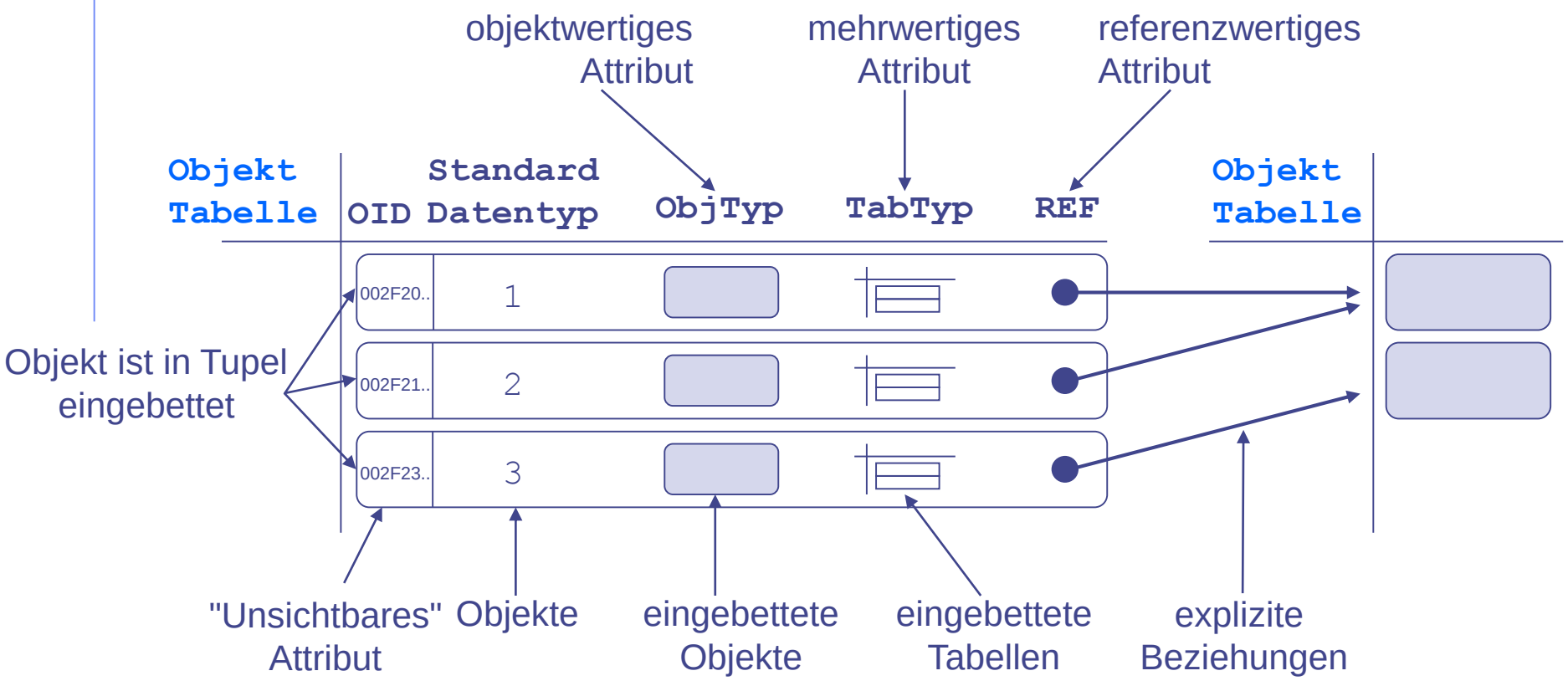
- Verwendung dieser Datentypen als Wertebereich von Attributen zur Konstruktion
  - weiterer benutzerdefinierter Datentypen sowie
  - herkömmlicher Tupeltabellen

Einführung 2/4
OO Konzepte im Oracle Datenmodell

- Nutzung eines Objekttyps auf zwei Arten
  - als Typ einer Tabellenspalte
  - als Typ einer Tabelle → **Objekttabelle**
- **Objekttabelle** (zusätzlich zu Tupeltabelle):
  - verwaltet Objekte (mit OID) statt Tupel (kein OID)
  - verwendet Objekttyp zur Definition von Struktur und Verhalten aller Objekte in einer Objekttabelle
  - dadurch konzeptionelle Trennung zwischen
    - Intension (Objekttyp) und
    - Extension (Objekttabelle mit Instanzen der Objekttypen)

Einführung 3/4

OO Konzepte im Oracle Datenmodell



Einführung 4/4

OO Konzepte im Oracle Datenmodell

Objekttyp

```
CREATE OR REPLACE TYPE Kunden_Typ AS OBJECT (
  KNr              INTEGER,
  Name             VARCHAR2(20),
  Adresse          Adress_Typ,      -- eingebetteter Objekttyp
  Telefonliste     Telefon_VArray,  -- mehrwertiges Attribut
  Auftraege        Auftrag_Tab,     -- eingebettete Tabelle
  -- Methoden
  MEMBER PROCEDURE AuftragErteilen (ANr INTEGER, l Lierferant_Typ));
```

Typdefinition

```
CREATE OR REPLACE TYPE BODY Kunden_Typ AS (
    MEMBER PROCEDURE AuftragErteilen (ANr INTEGER, ...)
    BEGIN ... END; );
```

Methodenimplementierung

Objekttabelle

```
CREATE TABLE Kunde OF Kunden_Typ
  NESTED TABLE Auftraege STORE AS Kundenauftraege;--Name für innere Tabelle,
                                                  --in der tabellenwertige
                                                  --Spalte abgelegt wird
```

Kunde	OID	KNr	Name	Adresse	Telefonliste	Auftraege
	002F20..	10	'Spock'	A-4020 …	[07/1212,07/123]	ANrEingang: 1001, 1004
	002F21..	20	'Kirk'	A-4721 …	[01/1212]	ANrEingang: 4921

Inhalt

- Einführung und Motivation ✓
- Objektrelationale Konzepte ✓
- Objektrelationale Konzepte in Oracle
  - Einführung ✓
  - Benutzerdefinierte Typen, Tabellen und Vererbung
    - Objekttypen
    - Tabellentypen und Array-Typen
    - Änderung von Typdefinitionen
    - Tupeltabellen und Objekttabellen
    - Vererbung und Sichten
  - Beziehungen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Benutzerdefinierte Typen 1/6

Struktur von Objekttypen

- Erzeugen von Objekttypen:

```
CREATE [OR REPLACE] TYPE <Objekttyp>
    [AUTHID CURRENT_USER | DEFINER]
    AS OBJECT (
    <Attributvereinbarung>{, <Attributvereinbarung>}
    [,<MAP/ORDER-Methodendeklaration>]
    {,<Methodendeklaration>})
  [[NOT] FINAL]
  [[NOT] INSTANTIABLE];

<Attributvereinbarung>::=<Attribut><Domäne>
```

- Zu Objekttypen lassen sich keine Initialisierungen (`DEFAULT`) oder Integritätsbedingungen wie `NOT NULL` definieren.
- Keine strenge Einkapselung realisiert, da auf Daten auch ohne Methoden zugegriffen werden kann

Benutzerdefinierte Typen 2/6
Vergleichsoperatoren für Objekttypen

- Es existieren **vordefinierte Vergleichsoperatoren** für Standarddatentypen, nicht jedoch für benutzerdefinierte Typen
- Daher werden **zwei Methodenarten** (`MAP` und `ORDER`) zur Verfügung gestellt, mit denen sich der **Benutzer selbst Ordnungsrelationen zwischen Objekten eines bestimmten Objekttyps** definieren kann
- Implementiert der Benutzer **keine solche Methode** zu einem Objekttyp, erfolgt **automatisch** ein **Werte-vergleich**
- Zu einem Objekttyp darf es nur **eine `ORDER` oder eine `MAP` Methode** geben

Benutzerdefinierte Typen 3/6
Vergleichsoperatoren für Objekttypen

```
MAP MEMBER FUNCTION <Map_Funktion> RETURN <Datentyp>
```

- damit kann eine **Abbildung eines Objektes auf einen Wert eines Standarddatentyps** spezifiziert werden, wodurch Oracle‘s vordefinierte Vergleichsoperatoren zum Einsatz kommen können
- bei Vergleichen der Art `<Objekt1> < <Objekt2>` wird sie **implizit aufgerufen** und der Vergleich quasi als `<Objekt1>.<Map_Funktion>() < <Objekt2>.<Map_Funktion>()` auf den berechneten Werten durchgeführt
- empfehlenswert für Sortierungen (`ORDER BY`-Klausel)

```
ORDER MEMBER FUNCTION <Order_Funktion>
    (<Var> IN <ObjektTyp>) RETURN INTEGER
```

- damit können **Vergleiche direkt durchgeführt werden**, wobei das zu vergleichende Objekt als Parameter übergeben wird
  - Rückgabewert = 0: `<Objekt> = <Var>`
  - Rückgabewert < 0: `<Objekt> < <Var>`
  - Rückgabewert > 0: `<Objekt> > <Var>`

Benutzerdefinierte Typen 4/6

Beispiel

```
CREATE TYPE Adress_Typ
  AS OBJECT (
    Plz          NUMBER(5),
    Ort          VARCHAR2(20),
    Strasse      VARCHAR2(20),
    Hausnr       VARCHAR2(5)
)
NOT FINAL;
```

```
CREATE TYPE Auftrag_Typ
  AS OBJECT (
    ANr          INTEGER,
    Eingang      DATE,
    ...
)
NOT FINAL;
```

Benutzerdefinierte Typen 5/6
Tabellen- und Array-Typen

- Array-Typen haben im Vergleich zu Tabellentypen
  - eine Ordnung (Zugriff über Positionsnummer möglich) und
  - eine fixe Elementanzahl `<max>`
  - Array-Werte können nicht direkt in SQL verarbeitet werden (können nur als Ganzes gelesen/gespeichert werden) – es muss eine Umwandlung in Tabellen erfolgen (`CAST`-Operator), wodurch jedoch Array-spezifische Funktionalität verloren geht

```
CREATE [OR REPLACE] TYPE <Tabellentyp> AS TABLE OF <Typname>;
```

```
CREATE OR REPLACE TYPE Auftrag_Tab AS TABLE OF REF Auftrag_Typ;
```

```
CREATE [OR REPLACE] TYPE <VAarry-Typ> AS VARRAY(<max>)OF <Typname>;
```

```
CREATE OR REPLACE TYPE Telefon_VArray AS VARRAY(3) OF VARCHAR2(20);
```

Typname kann sein:
Standarddatentyp (CHAR, NUMBER etc.)
Objekttyp, Referenztyp
Tabellentyp, Array-Typ

Benutzerdefinierte Typen 6/6
Änderung von Typdefinitionen

- Löschen von Typen: **`DROP TYPE`** `<Typ> [FORCE|VALIDATE];`
  - beim Löschen eines Typs dürfen keine Abhängigkeiten zu diesem Typ existieren
  - Löschen kann über `FORCE` dennoch erzwungen werden – die Folge sind Inkonsistenzen im Schema!
  - mit `VALIDATE` wird der Typ nur dann gelöscht, wenn keine Instanzen (auch nicht in Attributen die als Datentyp einen Supertyp haben) existieren
- Ändern von Typen: **`ALTER TYPE`** `<Typ>`

```
ADD ATTRIBUTE | METHOD ...[CASCADE | INVALIDATE];
DROP ATTRIBUTE | METHOD ...[CASCADE | INVALIDATE];
MODIFY ATTRIBUTE ...[CASCADE | INVALIDATE];
REPLACE AS OBJECT ...[CASCADE | INVALIDATE];
COMPILE [ SPECIFICATION | BODY ];
```

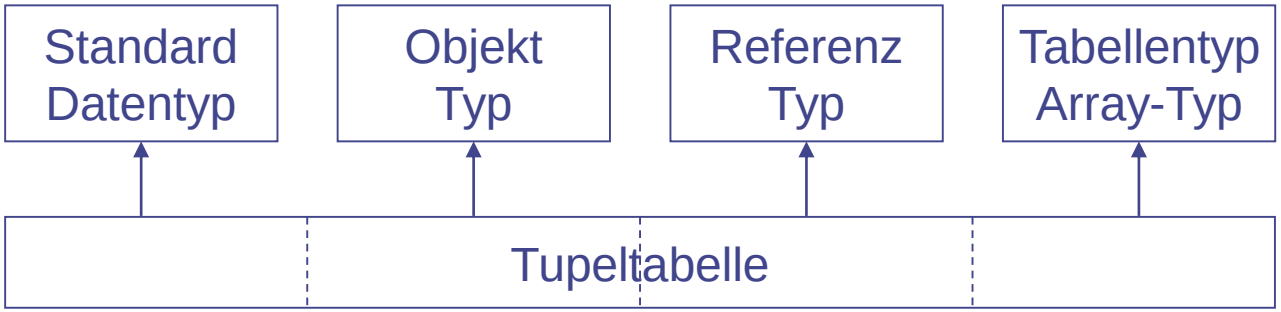
Tabellen

Tupeltabellen 1/2

- Definition einer Tupeltabelle

```
CREATE TABLE <Tabelle> (
  <Attributvereinbarung>{, <Attributvereinbarung>}
  {,<Tabellenbedingung>})
  [ {<STORE AS-Klausel>}]
  [ {<COLUMN-Klausel>}];

<Attributvereinbarung>::=<Attribut><Domäne>
  [DEFAULT <Ausdruck>] [<Attributbedingung>]
```



Legende:

→ …. „Attribut kann als Wertebereich haben“

Tabellen
Tupeltabellen 2/2

Kunde	KNr	Name	Adresse	Telefonliste	Aufträge
	10	'Spock'	A-4020 …	(0732/1212, 07762/123)	ANr Eingang ...
	20	'Kirk'	A-4721 …	(0732/3435)	ANr Eingang ...

```
CREATE TABLE Kunde (
    KNr             INTEGER,
    Name            VARCHAR2(20),
    Adresse         Adress_Typ,
    Telefonliste    Telefon_VArray,
    Auftraege       Auftrag_Tab )
    NESTED TABLE Auftraege STORE AS Kundenauftraege;
```

Geschachtelte Tabelle

Interne Speichertabelle zur Ablage der Tupel der tabellenwertige Spalte `Auftraege`

- Die Tupel aller eingebetteten Tabellen eines Attributes werden in einer separaten Tabelle verwaltet
- Mittels der `STORE AS`-Klausel muss dieser Tabelle ein Name zugewiesen werden

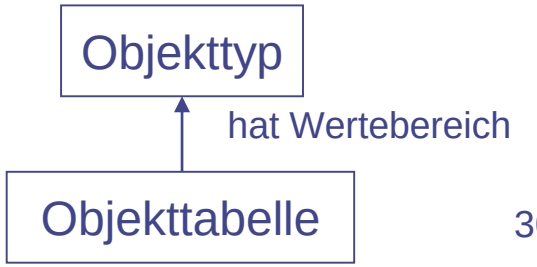
Tabellen
Objekttabellen 1/2

- Definition einer Objekttabelle

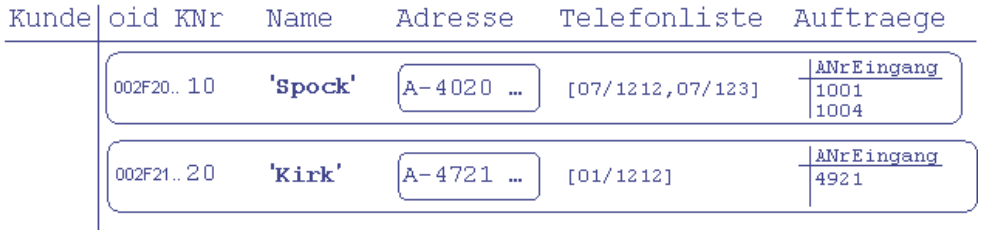
```
CREATE TABLE <ObjektTabelle> OF <ObjektTyp>
  [ <SUBSTITUTABLE-Klausel> ]
  [ (<Integritätsbedingung>{,<Integritätsbedingungen>}) ]
  [ <OID-Klausel> ]
  [ {<STORE AS-Klausel>}] /*für tabellenwertige Attribute*/
  [ {<COLUMN-Klausel>}] ;

<Integritätsbedingung>::=
  <Attribut> [ DEFAULT <Ausdruck> ] [ <Attributbedingung> ]
  [ <Tabellenbedingung> ]
```

- Ein Objekttyp definiert Struktur und Verhalten für ein oder mehrere Objekttabellen (nicht möglich mit Tabellentypen und Array-Typen!)



Tabellen
Objekttabellen 2/2




```
CREATE OR REPLACE TYPE Kunden_Typ
AS OBJECT (
   KNr           INTEGER,
   Name          VARCHAR2(20),
   Adresse       Adress_Typ,
   Telefonliste  Telefon_VArray,
   Auftraege     Auftrag_Tab);
```

```
CREATE TABLE Kunde OF Kunden_Typ (
  KNr     PRIMARY KEY,
  Name    NOT NULL
  )
  OBJECT IDENTIFIER IS PRIMARY KEY
  NESTED TABLE Auftraege STORE AS Kundenauftraege;
```

Angabe von Integritätsbedingungen

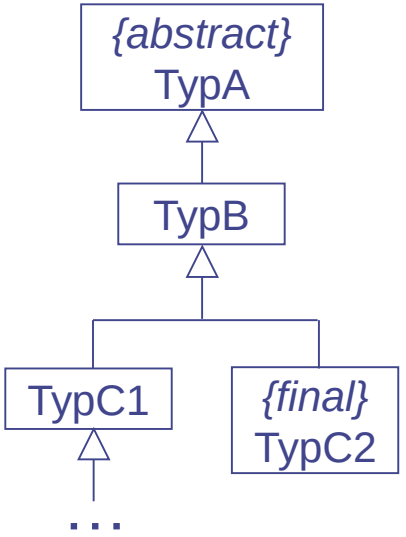
interne Speichertabelle

- Jedem Objekt einer Objekttabelle wird beim Einfügen in die Objekttabelle eine eindeutige, unveränderliche OID zugewiesen (eingebettete Objekte od. Objekte in mehrwertigen Attributen erhalten keinen OID)

  OBJECT IDENTIFIER IS
  - SYSTEM GENERATED (Voreinstellung)
  - PRIMARY KEY (OID wird vom Primärschlüssel abgeleitet)

- Alle Arten von DML-Statements, Integritätsbedingungen und Trigger, die auf Tupeltabellen anwendbar sind, sind auch auf Objekttabellen anwendbar!

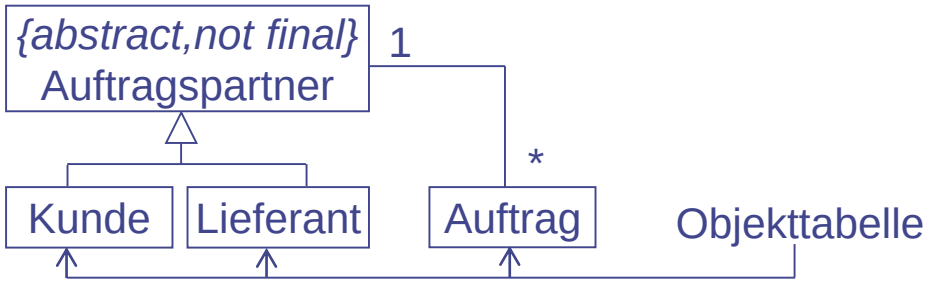
Vererbung 1/6
Subtypen




```
CREATE [OR REPLACE] TYPE <ObjektTyp> UNDER <Obertyp>
  AS OBJECT ( ... )
[[NOT] FINAL] [[NOT] INSTANTIABLE];
```

- Subtyp erbt Attribute u. Methoden des Obertyps und kann zusätzliche Attribute u. Methoden aufweisen
- Überschreiben von Methoden, Auswahl zur Laufzeit
- KEINE Mehrfachvererbung
- Einschränkungen
  - `FINAL`: keine Subtypen spezifizierbar (Voreinstellung)
  - `NOT INSTANTIABLE`: abstrakter Typ, keine Objekte instanziierbar
  - `FINAL u. NOT INSTANTIABLE` schließen sich gegenseitig aus
  - Änderung dieser Eigenschaften:
    - `ALTER TYPE <ObjektTyp> [NOT] FINAL;`
    - `ALTER TYPE <ObjektTyp> [NOT] INSTANTIABLE;`

Vererbung 2/6
Subtypen




```
CREATE OR REPLACE TYPE Auftragspartner_Typ AS OBJECT (
  Nr INTEGER,
  Name VARCHAR2(20), ...
  NOT FINAL MEMBER PROCEDURE AuftragErteilen(ANr INTEGER))
NOT FINAL NOT INSTANTIABLE;
```

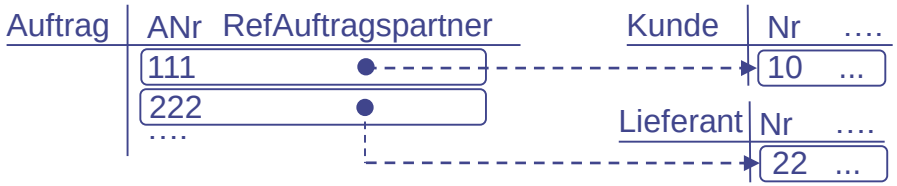
```
CREATE OR REPLACE TYPE Kunde_Typ UNDER Auftragspartner_Typ (
  Bonusstufe CHAR(1), ...
  OVERRIDING MEMBER PROCEDURE AuftragErteilen(ANr INTEGER));
```

```
CREATE OR REPLACE TYPE Lieferant_Typ UNDER Auftragspartner_Typ (
  Vorrat Vorrat_Tab, ...
  OVERRIDING MEMBER PROCEDURE AuftragErteilen(ANr INTEGER));
```

```
CREATE OR REPLACE TYPE Auftrag_Typ AS OBJECT (
  ANr INTEGER,
  ...
  Ref_Auftragspartner REF Auftragspartner_Typ );
```

```
CREATE TABLE Auftrag OF Auftrag_Typ;
CREATE TABLE Kunde OF Kunde_Typ;
CREATE TABLE Lieferant OF Lieferant_Typ;
```

Vererbung 3/6
Subtypen



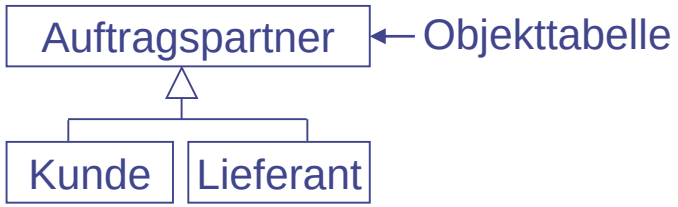
- Subtypobjekte können überall auftreten, wo Objekte eines Obertyps erlaubt sind (Polymorphismus)
  - Objekt-, referenz- u. kollektionswertige Attribute können Objekte unterschiedlicher Typen enthalten

```
CREATE OR REPLACE TYPE Auftrag_Typ AS OBJECT (
  ANr INTEGER, ...
  RefAuftragspartner REF Auftragspartner_Typ);

CREATE TABLE Auftrag OF Auftrag_Typ;
```

  - Objekttabelle des Obertyps kann auch Objekte des Subtyps enthalten

```
CREATE TABLE Auftragspartner
OF Auftragspartner_Typ;
```



Vererbung 4/6
Subtypen/Substitution

Annahme:
Auftraggeber_Typ ist nicht abstrakt (also INSTANTIABLE)

```
CREATE OR REPLACE TYPE Auftrag_Typ AS OBJECT (
  ...
  Auftragspartner Auftragspartner_Typ );
```

- Einschränkung auf homogene Objektmengen (beim `CREATE TABLE`-Statement)
  - zur Objekttabelle: `[NOT SUBSTITUTABLE AT ALL LEVELS-Klausel]`

```
CREATE TABLE Auftragspartner OF Auftragspartner_Typ
NOT SUBSTITUTABLE AT ALL LEVELS;
```

  - zu objektwertigem Attribut: `[<COLUMN-Klausel>`

```
COLUMN <Objektattribut> { NOT SUBSTITUTABLE AT ALL LEVELS |
                          IS OF (ONLY <Objekttypname>) }
```

```
CREATE TABLE Auftrag OF Auftrag_Typ
COLUMN Auftragspartner IS OF (ONLY Kunde_Typ);
```

  - zu tabellenwertigem Attribut:
    `NESTED TABLE <Attribut> [<COLUMN-Klausel>] STORE AS <Speichertabelle>`
  - zu Attributen vom Typ VARRAY:
    `VARRAY <Attribut> [<COLUMN-Klausel>] [STORE AS <Lob>]`
  - zu referenzwertigem Attribut: nicht möglich

Vererbung 5/6
Methoden

- Definition
  `[OVERRIDING][[NOT] FINAL][[NOT] INSTANTIABLE]`
  `<Methodendeklaration>`

- Überschreiben von Methoden – `OVERRIDING`
  gleicher Methodenname, gleiche Signatur
  - ⇨ Ersetzen einer vom Obertyp geerbten Methode; Voraussetzung für dynamische Bindung
- Dynamische Bindung
  - Feststellung des dynamischen Typs zur Laufzeit (im Gegensatz zur Prüfung des statischen Typs zur Kompilierzeit) u.
  - Ausführung der entsprechenden Methode (wenn Methode überschrieben wurde)
- Überladen von Methoden (unabhängig von Vererbung) gleicher Methodenname, geänderte Signatur
  - ⇨ neue Methode

Vererbung 6/6
Methoden

- Einschränkungen
  - `FINAL`: kein Überschreiben in Subtypen (Voreinstellung: `NOT FINAL`)
  - `NOT INSTANTIABLE`: abstrakte Methode, keine Implementierung, Objekttyp muss auch abstrakt sein

```
CREATE OR REPLACE TYPE Auftragspartner_Typ AS OBJECT (
  ANr INTEGER, ...
  NOT FINAL MEMBER PROCEDURE AuftragErteilen(ANr INTEGER))
  NOT FINAL;
```

```
CREATE OR REPLACE TYPE Kunde_Typ UNDER Auftragspartner_Typ
  (Bonusstufe CHAR(1), ...
  OVERRIDING MEMBER PROCEDURE AuftragErteilen(ANr INTEGER));
```

- `MAP`- u. `ORDER`-Methoden
  - `MAP`-Methoden sind in Subtypen erlaubt, wenn bereits im Obertyp (bzw. Wurzel der Typhierarchie) eine `MAP`-Methode spezifiziert wurde
  - `ORDER`-Methode darf nur in der Wurzel der Typhierarchie definiert werden

Sichten 1/4

Tupelsichten versus Objektsichten

- **Sicht** (virtuelle Tabelle) = **anwenderspezifische Darstellung** der in ein oder mehreren (Basis)Tabellen enthaltenen **Daten** auf Basis einer `SELECT`-Anweisung
- **Tupelsichten** können **als Attributwerte** eines Tupels **Objekte**, **Referenzen** und **Tabellen/Arrays** enthalten
- **Objektsichten** enthalten anstatt Tupel **Objekte** (virtuelle Objekttabellen) auf Basis von Tupeltabellen, Tupelsichten, Objekttabellen u. Objektsichten
- Durch diese neue Art von Sichten können **relationale Daten** von neuen Applikationen **objektorientiert verarbeitet** werden, **ohne existierende „Legacy“ Applikationen zu beeinträchtigen**
- **Der Zugriff erfolgt wie bei normalen Tabellen** - zur Umgehung der Update-Einschränkung bei mehreren Basistabellen können `INSTEAD OF`-Trigger verwendet werden

Sichten 2/4
Objektsichten

- Erzeugen von Objektsichten

```
CREATE [OR REPLACE][FORCE] VIEW <Sichtname>
  OF <Objekttyp>
  [WITH OBJECT IDENTIFIER {(<Spaltenliste>) | DEFAULT}]
  AS <SFW-Block>
  [WITH {READ ONLY | CHECK OPTION}];
```

  - `FORCE`: Sicht kann erzeugt werden, auch wenn Basistabellen/-sichten noch nicht existieren
  - `WITH OBJECT IDENTIFIER`: Legt für objektgenerierende Objektsichten fest, aus welchen Spalten der OID generiert werden soll

Sichten 3/4

Objektsicht auf Tupeltabelle ⇨ objekt**generierende** Sicht

Tupeltabelle

```
CREATE TABLE Kunde (
    KNr           INTEGER PRIMARY KEY,
    Name          VARCHAR2(20),
    Adresse       Adress_Typ,
    Telefonliste  Telefon_VArray);
```

Objekttyp

```
CREATE OR REPLACE TYPE Kunden_Typ AS OBJECT (
    KNr           INTEGER,
    Name          VARCHAR2(20),
    Adresse       Adress_Typ,
    Telefonliste  Telefon_VArray);
```

Objektsicht

```
CREATE OR REPLACE VIEW LinzKunde OF Kunden_Typ
WITH OBJECT IDENTIFIER (KNr)
  AS SELECT KNR, Name, Adresse, Telefonliste
  FROM Kunde k
  WHERE k.Adresse.Ort = 'Linz';
```

⇨ Übergang von relational zu objektrelational durch Objektsichten!

Sichten 4/4

Objektsicht auf Objekttabelle ⇨ objekt**erhaltende** Sicht

Objekttyp

```
CREATE OR REPLACE TYPE Kunden_Typ AS OBJECT (
  KNr              INTEGER,
  Name             VARCHAR2(20),
  Adresse          Adress_Typ,
  Telefonliste     Telefon_VArray,
  Auftraege        Auftrag_Tab);
```

Objekttabelle

```
CREATE TABLE Kunde OF Kunden_Typ (
  NESTED TABLE Auftraege STORE AS Kundenauftraege);
```

Objektsicht

```
CREATE OR REPLACE VIEW LinzKunde OF Kunden_Typ
  AS SELECT *
  FROM Kunde k
  WHERE VALUE(k) IS OF (ONLY Kunden_Typ)
  AND k.Adresse.Ort = 'Linz';
```

Objektrelationale Konzepte

- Einführung und Motivation ✓
- Objektrelationale Konzepte ✓
- Objektrelationale Konzepte in Oracle
  - Einführung ✓
  - Benutzerdefinierte Typen, Tabellen und Vererbung ✓
  - Beziehungen
    - über Referenzen
    - über geschachtelte Tabellen/Arrays
    - über geschachtelte Tabellen und Referenzen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

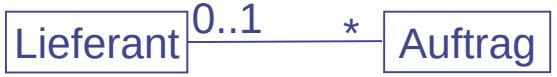
Beziehungen
… 1:m über Referenzen 1/2

- Die Vorteile von Referenzen gegenüber Fremdschlüsseln liegen in der direkteren ("natürlicheren") Modellierung und dem einfacheren (navigierenden) und effizienteren Zugriff
  - **Nachteil**: Dangling References (Prüfung mit `IS [NOT] DANGLING`)

- Die referenzierten Objekte müssen alle demselben Objekttyp angehören, können sich aber grundsätzlich auf mehrere Objekt(!)tabellen (dieses Objekttyps) verteilen

- Integritätsbedingung die Referenzen auf genau eine Objekttabelle einschränken
  - Tabellenbezogen:
    `[ SCOPE IS <ObjektTabelle> ]`
  - Attributbezogen:
    `[ SCOPE FOR (<RefAttr>) IS <ObjektTabelle>]`

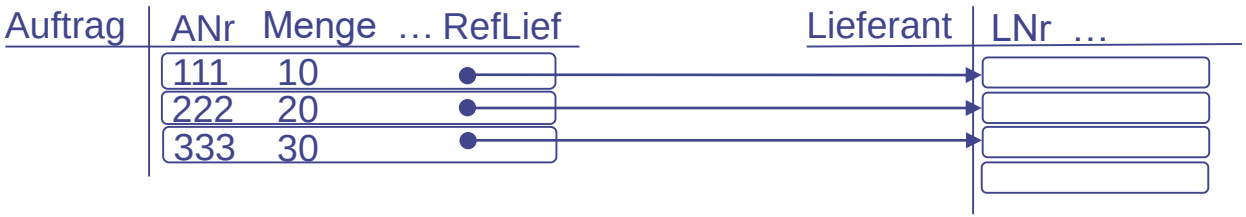
Beziehungen
… 1:m über Referenzen 2/2



- c:cm-Beziehung zwischen Lieferant und Auftrag:

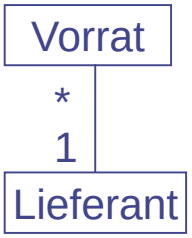
```
CREATE TYPE Lieferant_Typ AS OBJECT (...);
CREATE TYPE Auftrag_Typ AS OBJECT (
   ANr NUMBER(10), ...,
   RefLief REF Lieferant_Typ);

CREATE TABLE Lieferant OF Lieferant_Typ;
CREATE TABLE Auftrag OF Auftrag_Typ;
```



Beziehungen

… 1:m über geschachtelte Tabellen/Arrays 1/2



- Über geschachtelte Tabellen/Arrays können 1:m-Beziehungen direkt ausgedrückt werden
- 1:cm-Beziehung zwischen `Lieferant` und `Vorrat`:

```
CREATE TYPE Vorrat_Typ AS OBJECT (Anzahl NUMBER, ...,
    RefTeil REF Teil_Typ);
CREATE TYPE Vorrat_Tab AS TABLE OF Vorrat_Typ;
CREATE TYPE Lieferant_Typ AS OBJECT (LNr INTEGER, ...,
    Vorrat Vorrat_Tab);

CREATE TABLE Lieferant OF Lieferant_Typ
  NESTED TABLE Vorrat STORE AS LiefVorrat;
```



Beziehungen
… 1:m über geschachtelte Tabellen/Arrays 2/2

- Jedes `Vorrat`-Tupel hat einen eindeutigen Bezug zu seinem `Lieferanten`, da es über den Tabellentyp `Vorrat_Tab` in der Objekttabelle `Lieferant` eingebettet ist
  - ein Fremdschlüssel in `Vorrat`, der auf den `Lieferanten` verweist, ist nicht nötig
  - ein `Vorrat` ohne seinen `Lieferanten` kann nicht existieren
  - die `Vorrat`-Tupel sind keine Objekte und von außen nicht referenzierbar
  - wohl aber dürfen `Vorrat`-Tupel auf weitere Objekte über `REF` verweisen (z.B. referenziert jeder Vorrat über `RefTeil` ein entsprechendes Teil)

Beziehungen

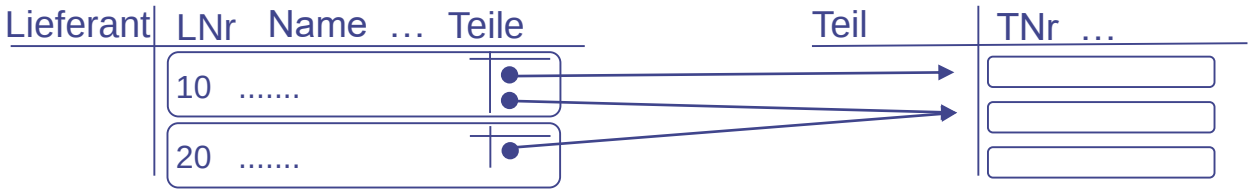
.. m:n über geschachtelte Tabellen/Arrays und Referenzen

- Geschachtelte Tabellen/Arrays werden mit Referenzen kombiniert, sodass nun die/der innere Tabelle/Array Referenzen anstelle von Tupeln enthält
- cm:cn-Beziehung zwischen `Lieferant` und `Teil`:




```
CREATE TYPE Teil_Typ AS OBJECT (...);
CREATE TYPE Teil_Tab AS TABLE OF REF Teil_Typ;
CREATE TYPE Lieferant_Typ AS OBJECT (LNr..., Teile Teil_Tab);

CREATE TABLE Lieferant OF Lieferant_Typ
   NESTED TABLE Teile STORE AS LieferantenTeile;
CREATE TABLE Teil OF Teil_Typ;
```

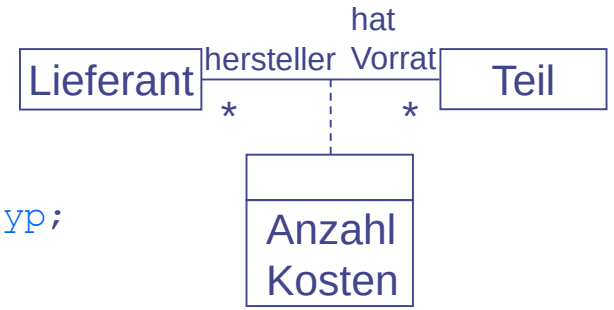


Beziehungen
… mit Beziehungsattributen

- Dazu sind geschachtelte Tabellen mit Referenzen nötig, bei denen die inneren Tabellen nicht nur die Referenzen, sondern auch die Attribute der Beziehung beinhalten

```
CREATE TYPE Teil_Typ AS OBJECT (...);
CREATE TYPE Vorrat_Typ AS OBJECT (
     Anzahl NUMBER (10),
     Kosten NUMBER (10,2),
     RefTeil REF Teil_Typ);
CREATE TYPE Vorrat_Tab AS TABLE OF Vorrat_Typ;
CREATE TYPE Lieferant_Typ AS OBJECT
     (... Vorrat Vorrat_Tab);

CREATE TABLE Teil OF Teil_Typ;
CREATE TABLE Lieferant OF Lieferant_Typ
   NESTED TABLE Vorrat STORE AS Vorratstabelle;
```



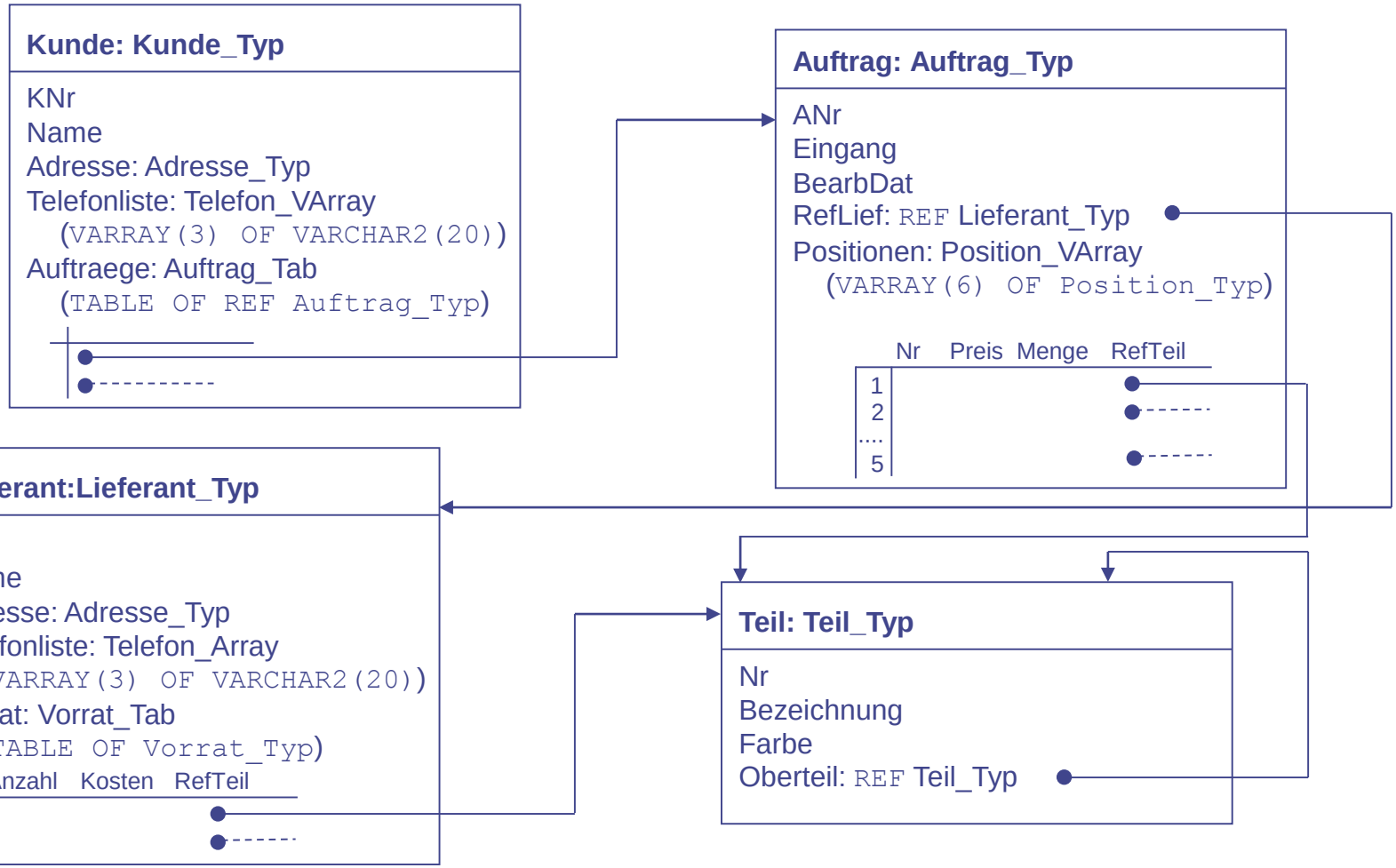


Beziehungen

Wann welche Beziehungsart?

	1:1-Beziehung	1:m-Beziehung	m:n-Beziehung
Fremd-schlüssel	falls Objekt sichtbar sein soll wird eine allg. 1:m-Beziehung realisiert	die zu-1-Richtung der Beziehung wird direkt modelliert	
Referenzen	die 1:1-Kardinalität muss durch Integritätsbedingungen überwacht werden		über eigene Beziehungstabelle
Geschachtelte Tabellen/Arrays	**[** das eingebettet Objekt (eigentlich Tupel) ist nicht sichtbar („private“) und kann an keiner weiteren Beziehung teilnehmen **]**	geschachtelte Tabelle/Array (mehrwertige Attribute) drücken die zu-m-Richtung direkt aus hier muss zwischen Tabellen und Arrays entschieden werden	
Geschachtelte Tabellen/Arrays und Referenzen		**[** prinzipiell Realisierung einer m:n-Beziehung 1:m müssen Integritäts-bedingungen überwachen **]**	geschachtelte Tabellen mit REFs sind die einzige Möglichkeit, um ohne Beziehungstabelle auszukommen

Benutzerdefinierte Typen, Tabellen und Beziehungen – Zusammenfassendes Beispiel



Schema in Oracle 1/4

```
CREATE TYPE Teil_Typ;          -- Vorwärtsdeklaration
CREATE TYPE Lieferant_Typ;     -- Vorwärtsdeklaration

CREATE TYPE Position_Typ AS OBJECT (
  Nr        NUMBER(4),
  Preis     NUMBER(12,2),
  Menge     NUMBER(6),
  RefTeil   REF Teil_Typ);

CREATE TYPE Position_Array AS VARRAY(6) OF Position_Typ;

CREATE TYPE Auftrag_Typ AS OBJECT (
  ANr           INTEGER,
  Eingang       DATE,
  Bearbdat      DATE,
  RefLief       REF Lieferant_Typ,
  Positionen    Position_Array);

CREATE TYPE Auftrag_Tab AS TABLE OF REF Auftrag_Typ;
```

Schema in Oracle 2/4

```
CREATE TYPE Adresse_Typ AS OBJECT (
  Plz       NUMBER(5),
  Ort       VARCHAR2(20),
  Strasse   VARCHAR2(20),
  Hausnr    VARCHAR2(5));

CREATE TYPE Telefon_Array AS VARRAY(3) OF VARCHAR(20);
CREATE TYPE Kunde_Typ AS OBJECT (
  KNr             INTEGER,
  Name            VARCHAR2(20),
  Adresse         Adresse_Typ,
  Telefonliste    Telefon_Array,
  Auftraege       Auftrag_Tab);

CREATE TYPE Teil_Typ AS OBJECT (
  Nr          NUMBER(10),
  Bezchng     VARCHAR2(20),
  Farbe       VARCHAR2(10),
  Vaterteil   REF Teil_Typ);
```

Schema in Oracle 3/4

```
CREATE TYPE Vorrat_Typ AS OBJECT (
  Anzahl       NUMBER(10),
  Kosten       NUMBER(10,2),
  RefTeil      REF Teil_Typ);

CREATE TYPE Vorrat_Tab AS TABLE OF Vorrat_Typ;

CREATE TYPE Lieferant_Typ AS OBJECT (
  LNr            INTEGER,
  Name           VARCHAR2(20),
  Adresse        Adresse_Typ,
  Telefonliste   Telefon_Array,
  Vorrat         Vorrat_Tab);
```

Schema in Oracle 4/4

```
-- Objekttabellen

CREATE TABLE Kunde OF Kunde_Typ (
  KNr     PRIMARY KEY,
  Name    NOT NULL)
  NESTED TABLE Auftraege STORE AS Kundenauftraege;

CREATE TABLE Auftrag OF Auftrag_Typ;

-- keine Tabelle zu Position
-- Position ist eingebettet in Auftrag

CREATE TABLE Lieferant OF Lieferant_Typ (
  LNr       PRIMARY KEY,
  Name      NOT NULL)
  NESTED TABLE Vorrat STORE AS LiefVorrat;

CREATE TABLE Teil OF Teil_Typ;
```

Objektrelationale Konzepte

- Einführung und Motivation ✓
- Objektrelationale Konzepte ✓
- Objektrelationale Konzepte in Oracle
  - Einführung ✓
  - Benutzerdefinierte Typen, Tabellen und Vererbung ✓
  - Beziehungen ✓
  - Erweitertes SQL zur Datenmanipulation
    - Konvertierungsfunktionen für Anfrageergebnisse
    - Pfadausdrücke
    - Eingebettete Tabellen
    - Konstruktoren und Zuweisungen
    - Methoden
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Erweitertes SQL
Konvertierungsfunktionen für Anfrageergebnisse

- Bei der Formulierung einer Anfrage ist es möglich festzulegen, welcher Art das Ergebnis sein soll:
  - ein Tupel aus den Attributwerten des Objekts
    `SELECT Attribut_1,...,Attribut_n FROM <ObjektTabelle>;`
  - ein Objekt aus einem Tupel
    `SELECT VALUE(<TAlias>) FROM <ObjektTabelle> <TAlias>;`
  - eine Referenz (den OID) auf das Objekt
    `SELECT REF(<TAlias>) FROM <ObjektTabelle> <TAlias>;`
  - ein Objekt zu einem OID
    `SELECT DEREF(<Pfad>) FROM <ObjektTabelle> <TAlias>;`
    wobei der letzte Teil des Pfades ein Referenzattribut sein muss
- Die Operationen `VALUE, REF, DEREF` können in `SELECT` und `WHERE`-Klauseln verwendet werden, jedoch NICHT innerhalb von Pfaden

```
CREATE TYPE Lieferant_Typ AS OBJECT
(
   LNr           INTEGER,
   Name          VARCHAR2(20),
   Adresse       Adresse_Typ,
   Telefonliste  Telefon_Array,
   Vorrat        Vorrat_Tab);
```

```
CREATE TABLE Lieferant OF Lieferant_Typ (
   LNr           PRIMARY KEY,
   Name          NOT NULL)
   NESTED TABLE Vorrat STORE AS LiefVorrat;
```

Erweitertes SQL

Konvertierungsfunktionen für Anfrageergebnisse

```
SELECT * FROM Lieferant;
```

LNR	NAME	ADRESSE(PLZ, ORT, STRASSE, HAUSNR)	TELEFONLISTE	VORRAT(ANZAHL, KOSTEN, REFTEIL)
1000	Altmann	ADRESSE_TYP (4040, 'Linz', 'Schwagerweg', 7')	TELEFON_ARRAY ('0732 879879', '0999 23423423')	VORRAT_TAB(VORRAT_TYP(10, 500, 00002202089B3032E501DB4081A3843A2270B321EC87F42B2961744ADEB44634806BE38898), VORRAT_TYP(10, 500, 00002202089B3032E501DB 4081A3843A2270B321EC87F42B2961744ADEB44634806BE38898), VORRAT_TYP(10, 500, 00002202089B3032E501DB4081A3843A2270B321EC87F42B2961744ADEB44634806BE38898))

```
SELECT VALUE(l) FROM Lieferant l;
```

VALUE(L)(LNR, NAME, ADRESSE(PLZ, ORT, STRASSE, HAUSNR), TELEFONLISTE, VORRAT(ANZAHL, KOSTEN, REFTEIL))
LIEFERANT_TYP(1000, 'Altmann', ADRESSE_TYP(4040, 'Linz', 'Schwagerweg', 7'), TELEFON_ARRAY('0732 879879', '0999 23423423'), VORRAT_TAB (VORRAT_TYP(10, 500, 00002202089B3032E501DB4081A3843A2270B321EC87F42B2961744ADEB44634806BE38898), VORRAT_TYP(10, 500, 00002202089B3032E501DB4081A3843A2270B321EC87F42 B2961744ADEB44634806BE38898), VORRAT_TYP(10, 500, 00002202089B3032E501DB4081A3843A2270B321EC87F42B2961744ADEB44634806BE38898)))

```
SELECT REF(l) FROM Lieferant l;
```

REF(L)
0000280209D57D260DA79F47B5BE3DC650ABF7BA62C584E0926F56414083EA56CA0C50759002C0034A0000

Erweitertes SQL
Pfadausdrücke 1/2

- Der Zugriff auf Attribute wurde ausgedehnt in einen Zugriff auch auf Objekte und Referenzen und daraus resultierenden Objekten und Referenzen etc.
  `<Pfad>::= <TAlias>{.<Obj/RefAttr>} [.<Attribut>]`
- Innerhalb (!) von Pfadausdrücken erfolgt implizit eine Dereferenzierung
  - Bestimme das eingebettete Adress-Objekt eines Kunden k:
    `SELECT k.Adresse FROM Kunde k WHERE ...`
  - Ermittle den Ort der eingebetteten Adresse:
    `SELECT k.Adresse.Ort FROM Kunde k WHERE ...`
  - Bestimme den OID des referenzierten Lieferanten:
    `SELECT a.RefLief FROM Auftrag a WHERE ...`
  - Berechne die `LNr` des referenzierten Lieferanten. Vom Auftrag a wird zum Lieferanten traversiert:
    `SELECT a.RefLief.LNr FROM Auftrag a WHERE ...`

Erweitertes SQL
Pfadausdrücke 2/2

```
CREATE TYPE Auftrag_Typ AS OBJECT (
   ANr             INTEGER,
   Eingang         DATE,
   Bearbdat        DATE,
   RefLief         REF   Lieferant_Typ,
   Positionen      Position_Array);
```

```
CREATE TABLE Auftrag OF Auftrag_Typ;
```

- Bestimme den Lieferanten als Objekt:
  ```
  SELECT DEREF(a.RefLief) FROM Auftrag a WHERE ...
  ```
- Bestimme alle Aufträge an den Lieferanten Scotti:
  ```
  SELECT a.ANr FROM Auftrag a WHERE a.RefLief.Name =
  'Scotti';
  ```
- Verbund über Referenzen:
  ```
  SELECT a.ANr FROM Auftrag a, Lieferant l
  WHERE a.RefLief=REF(l) AND l.Name='Scotti';
  ```

```
CREATE TYPE Auftrag_Typ AS OBJECT (
   ANr              INTEGER,
   Eingang          DATE,
   Bearbdat         DATE,
   RefLief          REF    Lieferant_Typ,
   Positionen       Position_Array);
```

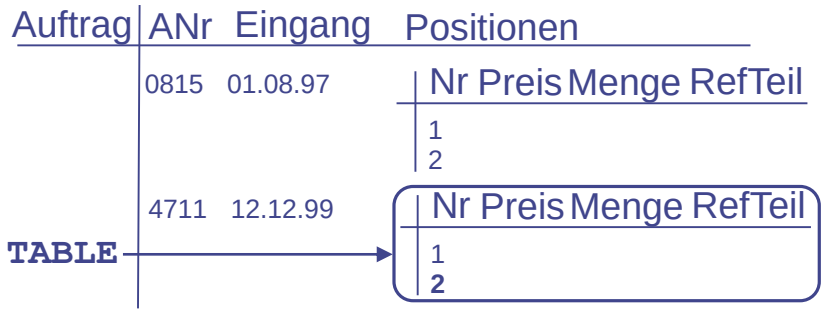
```
CREATE TABLE Auftrag OF Auftrag_Typ;
```

Erweitertes SQL

Qualifizierung einer eingebetteten Tabelle mit TABLE

- Die Klausel TABLE (<SFW-Block>) bietet die Möglichkeit, durch eine Art „Entschachtelung“ eingebettete Tabellen anzusprechen, z.B. um Tupel in diese einzutragen oder zu löschen
- Der SFW-Block hinter TABLE muss genau 1 Tupel identifizieren und darf nur 1 Attribut (mehrwertig!) des Tabellentyps selektieren
- Auf diese Weise wird genau 1 innere Tabelle angesprochen
- Ein entsprechend spezifizierter Tabellen-Alias bezieht sich auf alle Tupel dieser eingebetteten Tabelle

```
SELECT p.Nr, p.Menge
FROM TABLE ( SELECT a.Positionen
             FROM Auftrag a
             WHERE ANr=4711) p
WHERE Nr=2;
```




```
INSERT INTO TABLE (SELECT Vorrat FROM Lieferant WHERE LNr=7)
VALUES (6,100,NULL);
```

Erweitertes SQL

Bearbeitung mehrerer eingebetteter Tabellen mit `TABLE`

- Oftmals ist es nötig, jene Tupel einer äußeren Tabelle zu selektieren, deren eingebettete Tabellen ein gewisses Tupel enthalten
- Dazu werden über `TABLE (<Tabellen-Attribut>)` die Tupel der eingebetteten Tabellen angesprochen
- Bestimme alle Lieferanten, die Schrauben vorrätig haben:

```
SELECT l.LNr, l.Name
FROM Lieferant l
WHERE EXISTS
    ( SELECT *
      FROM TABLE (l.Vorrat) v
      WHERE v.RefTeil.Bezchng='Schraube');
```

Oder:

```
SELECT l.LNr, l.Name
FROM Lieferant l, TABLE (l.Vorrat) v
WHERE v.RefTeil.Bezchng='Schraube');
```

Erweitertes SQL
Konstruktion von Objekten und Kollektionen

- Objekte, Tabellen und VArrays können als Konstanten erzeugt werden, um z.B. objektwertige Attribute mit diesen zu belegen
- Zu diesem Zweck besitzt jeder benutzerdefinierte Typ automatisch einen Konstruktor
- Dieser hat denselben Namen wie der Typ und verwendet die Objektstruktur als Parameterliste

```
Adresse_Typ(12345, 'Vulkan', 'Enterprise Road', '9')

Position_VArray( Position_Typ(1,100,20,NULL),
                 Position_Typ(2,200,30,NULL),...)

INSERT INTO Lieferant (LNr, Name, Adresse, Vorrat)
VALUES ( 10, 'Scotti',
         Adresse_Typ(81730, ....),
         Vorrat_Tab(   Vorrat_Typ(1,100,NULL),
                       Vorrat_Typ(2,200,NULL)));
```

```
CREATE TYPE Kunde_Typ AS OBJECT (
   KNr              INTEGER,
   Name             VARCHAR2(20),
   Adresse          Adresse_Typ,
   Telefonliste     Telefon_Array,
   Auftraege        Auftrag_Tab);
```

```
CREATE TYPE Auftrag_Tab AS TABLE OF
REF Auftrag_Typ;
```

Erweitertes SQL

Zuweisungen von Objekten, Referenzen und Kollektionen

- Herkömmliches `UPDATE` wird so erweitert, dass Pfade, Objekte, Referenzen und Kollektionen als Ausdruck der rechten Seite erlaubt sind
  - Einem Auftrag wird ein Lieferant zugewiesen:

```
UPDATE Auftrag a
SET RefLief = (SELECT REF(l) FROM Lieferant l
                             WHERE l.LNr=7)
WHERE a.ANr=4711;
```

  - Das Objekt kann als Konstante gesetzt werden:

```
UPDATE Kunde k SET Adresse=Adresse_Typ(81730, .....)
WHERE Name='Scotti';
```

  -

```
UPDATE Kunde k
SET Auftraege = CAST(MULTISET(SELECT REF(a)
FROM Auftrag a WHERE ...) AS Auftrag_Tab);
```

Multimenge wird als Tabelle aufgefasst und anschließend mit CAST zu einem Auftrag_Tab-Typ konvertiert.

  -

```
UPDATE Kunde SET Auftraege = NULL;
```

Erweitertes SQL
Zuweisungen von Objekten, Referenzen und Kollektionen

- Bearbeiten von inneren Tabellen

```
UPDATE TABLE(SELECT Vorrat FROM Lieferant WHERE LNr=7)
SET Kosten = Kosten * 1.1
WHERE Kosten < 100;
```

- Ein einzelnes Objekt einer inneren Tabelle ist auch komplett ersetzbar

```
UPDATE TABLE(SELECT l.Vorrat FROM Lieferant l WHERE
l.LNr=1000) v
SET VALUE(v) = Vorrat_Typ(10, 500,
(SELECT REF(t) FROM Teil t WHERE t.Nr=30));
```

Erweitertes SQL
Aufruf von Methoden

- Methodenaufrufe können als Bestandteil der `SELECT` oder der `WHERE`-Bedingung verwendet werden
- Der Methoden-Aufruf wird mit dem Tabellen-Alias qualifiziert
- Auftragsvolumen aller Aufträge mit mehr als 4 Positionen:

```
SELECT a.BerechneAuftragsvolumen()
FROM Auftrag a
WHERE a.Anzahl()>4;
```

- Eine Methode kann auch zu einem referenzwertigen Attribut aufgerufen werden (dabei wird wieder implizit dereferenziert)
  `SELECT <TAlias>.<ReferenzAttr>.<Methode>(...)...`
- Es sind nur Methoden ohne Seiteneffekte innerhalb eines SQL-Statements ausführbar

Erweitertes SQL
Typhierarchien

- Typprüfung von Objekten zur Laufzeit: `IS [NOT] OF`
  - Typ od. Subtypen: `<Ausdruck> IS [NOT] OF (<Typ>{, <Typ>})`
  - Typ ohne Subtypen: `<Ausdruck> IS [NOT] OF ( ONLY <Typ> )`
  - ```
    SELECT VALUE(obj) FROM SichtA obj
    WHERE VALUE(obj) IS OF (TypA);
    ```
  - `-"- WHERE VALUE(obj) IS OF (ONLY TypA); -- genau dieser TypA`
- Typkonvertierung: `TREAT`
  - `TREAT (<Objekt> AS <Typ>)`
  - `TREAT (<Referenz> AS REF <Typ>)`
  - wenn `<Typ>` nicht der aktuelle Typ oder ein Obertyp ist, wird NULL geliefert
  - ```
    SELECT TREAT(VALUE(obj) AS TypB).<BAttribut>
    FROM SichtA obj
    WHERE VALUE(obj) IS OF (TypB);
    ```

Erweitertes SQL
Zusammenfassung

- In der `FROM`-Klausel sind zusätzliche Anweisungen erlaubt:
  - `TABLE (<SFW-Block>) <TAlias>` (Qualifikation einer eingebetteten Tabelle)
- Als Ausdruck der `SELECT`-Klausel ist erlaubt:
  - `CAST (...) AS <Typ>` (Typkonvertierung)
  - `<TAlias>.<Attr>` für mehrwertige Attribute
  - `TREAT` zur Typkonvertierung (`IS OF (<Type>)` zur Typprüfung) bei Typhierarchien
- Zur Bildung von Ausdrücken in `SELECT` oder in `WHERE`-Bedingungen sind neue Möglichkeiten erlaubt
  - `<TAlias>.<Attr>` für objekt- und referenzwertige Attribute
  - Konvertierungsfunktionen `REF, DEREF` und `VALUE`
  - Typkonstruktoren (für Objekte, Tabellen und VArrays)
  - Methodenaufrufe

Objektrelationale Konzepte

- Einführung und Motivation ✓
- Objektrelationale Konzepte ✓
- Objektrelationale Konzepte in Oracle ✓
  - Einführung
  - Benutzerdefinierte Typen, Tabellen und Vererbung
  - Beziehungen
  - Erweitertes SQL zur Datenmanipulation
- Zusammenfassung
- Anhang I: Objektrelationale Konzepte in SQL:2008
- Anhang II: Sichthierarchien

Zusammenfassung

Vorteile objektrelationaler DBMS

- Einfachere Abbildung zwischen Datenbank- und Anwendungsobjekte
  - ⇨ OO-Modellierung
- Unterstützen mehr Semantik und Funktionalität in der Datenbank (z.B. explizites Wissen über Spezialisierungsbeziehungen)
  - ⇨ mächtigere SQL-Anfragen
- ORDBMS auch als RDBMS verwendbar ("aufwärtskompatibel")

Zusammenfassung
Nachteile objektrelationaler DBMS

- Objektrelationale Erweiterungen weder in SQL:2008 noch in aktuellen System konsequent genug spezifiziert bzw. umgesetzt
  - Unterschiedliche Datenmodelle von SQL:2003, DB2, Oracle, Informix und Postgres
  - Syntax und Semantik der Datenbanksprachen nicht immer intuitiv (im Sinne der Objektorientierung)
  - Datenmodellkomplexität und -heterogenität erschweren Datenbank-Migration

Zusammenfassung
Ausblick

- Marktführer (Oracle, DB2) setzen auf objektrelationale Erweiterungen
- Weiterentwicklung von Entwurfswerkzeugen, die objektrelationale Datenbankschemata generieren bzw. den Entwurf solcher begleiten
  - Aufgrund der Mächtigkeit objektrelationaler Datenmodelle ergeben sich mehrere Möglichkeiten zur Modellierung eines Sachverhalts
  - Güte eines Schemas: Metriken?
    - Verständlichkeit (Anzahl der Tabellen, Tabellenhierarchietiefen, Schachtelungstiefen etc.)
    - Performance bei bestimmten Anfragearten

Anhang I

Objektrelationale Konzepte SQL:2008

SQL:2008
Neue Datentypen

- `BLOB, CLOB`: Datentypen für große Binär-/Zeichenkettenobjekte (`binary/character large objects`)
- `BOOLEAN`: boolesche Werte `TRUE, FALSE, UNKNOWN`
- `ARRAY`: Kollektionstyp für mehrwertige Attribute (Ordnung!)
- `MULTISET`: Kollektionstyp für ungeordnete Multimengen (mit Duplikaten)
- `REF`-Typen: zur eindeutigen Identifikation von Tupeln (OID) sowie zur Navigation über Pfadausdrücke
- `ROW`-Typen: Typkonstruktor für strukturierte Attribute
- benutzerdefinierte Datentypen: Konstruktion neuer Typen
- ...

SQL:2008

`BLOB` und `CLOB`

- Speicherung großer Binärfolgen/Zeichenketten
  - Bilder, Audio-/Videodaten, XML-Dokumente, . . .
- Angabe der max. Größe erforderlich
- Grenzen sind systemspezifisch

```
CREATE TABLE Einwohner (
  ...
  Foto BLOB(100K));
```

- erfordern spezielle Behandlung beim Zugriff
- Einschränkungen:
  - kein Primär-/Fremdschlüssel, keine Gruppierung/Sortierung
  - keine Mengen-, Verbund- und Größer/Kleiner-Operatoren

SQL:2008
`ARRAY`-Typkonstruktor

- Datentypen zur Speicherung mehrerer Werte für ein Attribut
- `ARRAY`-Typkonstruktor

```
<datentyp> ARRAY[k]
```

- keine Schachtelung möglich!
- Operatoren
  - Konstruktor: `ARRAY[elementliste]`
  - Direkter Elementzugriff: `myArray[2]`
  - Entschachtelung: `UNNEST`-Operator transformiert Array in Tabelle
  - Konkatenation zweier Arrays: `||`
  - ...

SQL:2008
ARRAY-Operationen

- Definition von ARRAY-Attributen

```
CREATE TABLE Einwohner (
  name VARCHAR(50),
  vorname VARCHAR (50),
  geburtstag DATE,
  foto BLOB(100K),
  telefone VARCHAR(20) ARRAY[5]
);
```

- Erzeugung von ARRAY-Werten über Array-Konstruktor

```
INSERT INTO Einwohner (vorname, name, telefone)
     VALUES ('Ann', 'Joy',
        ARRAY ['01234567', '034466576']);
```

- Überschreiben von ARRAY-Werten

```
UPDATE Einwohner
SET telefone =  ARRAY ['04234567']
WHERE name = 'Pause';
```

SQL:2008
ROW-Typkonstruktor

- Typkonstruktor für strukturierte Attribute
- anonym ⇨ kein expliziter Typname

```
CREATE TABLE Einwohner (
  name VARCHAR(50),
  vorname VARCHAR (50),
  adresse ROW ( strasse VARCHAR (30),
                plz CHAR (5),
                ort VARCHAR (30))
);
```

- Operatoren
  - Konstruktor: `ROW[werteliste]`
  - Zugriff auf Tupelfeld: `adresse.ort`
  - Vergleich zweier Tupel: `<,>,=, <>, …`
  - etc.

SQL:2008
ROW-Operationen

- Konstruktion durch Tupelkonstruktor ROW()

```
INSERT INTO Einwohner (vorname, name, adresse)
   VALUES ('Ann', 'Joy'),
     ROW ('Schwagerweg 7', '4040', 'Linz'));
```

- Zugriff auf Tupelfeld

```
SELECT adresse.ort
FROM Einwohner;
```

SQL:2008
MULTISET-Typkonstruktor

- multimengenwertige Spalten
- ungeordnete, mit Duplikaten

```
<datentyp> MULTISET
```

- Operatoren
  - Konstruktor: MULTISET(werteliste), …
  - Entschachtelung: UNNEST-Operator transformiert Multiset in Tabelle
  - Duplikateliminierung, Elementtest, Vereinigung, Differenz, etc.
- Beispiel

```
CREATE TABLE Einwohner (
  name VARCHAR(50),
  vorname VARCHAR (50),
  adresse ROW ( strasse VARCHAR (30), plz CHAR (5),
                ort VARCHAR (30)) MULTISET,
  telefonNr VARCHAR (20) MULTISET);
```

SQL:2008
Benutzerdefinierte Datentypen

- Unterscheidung in
  - **Distinct-Typen:** Definition eines Datentyps direkt als „Umbenennung“ eines Basisdatentyps
  - **Strukturierte Typen:** Typkonstruktor für benannte strukturierte Datentypen

SQL:2008
Distinct-Typen

- Benutzerdefinierter Datentyp, der eine Kopie eines Basisdatentyps unter einem neuen Namen ablegt.
  - Wiederverwendung
  - Strenge Typisierung (erhöht Typsicherheit)
  - Nicht-optionales Schlüsselwort `FINAL` schließt Subtypbildung aus
  - Systemdefinierte Vergleichsoperatoren basierend auf dem Quelltyp
  - Cast-Operatoren zur Konversion zwischen Distinct- und Quelltyp

```
CREATE TYPE METER AS INTEGER FINAL;
CREATE TYPE QUADRATMETER AS INTEGER FINAL;
```

- kein Vergleich/Zuweisung von Attributen verschiedener Distinct-Typen (auch wenn gleicher Basisdatentyp)

```
CREATE TABLE Rechteckt (
   länge METER, breite METER,
   fläche QUADRATMETER); --breite und fläche sind inkompatibel
```

SQL:2008
Strukturierte Typen und Vererbung

- Benutzerdefinierter Datentyp, der einem Objekttyp mit Attributen und Methoden entspricht.
- Ein strukturierter Typ heißt Wurzeltyp, wenn er keinen Supertyp besitzt.

```
CREATE TYPE <typname>
AS (Attributdefinitionsliste)
[[NOT] INSTANTIABLE]
[NOT] FINAL
[Referenzgenerierung [Referenzcasting]]
[Methodendeklarationsliste]
```

  - Optional: Instanziierung verbieten (abstrakter Typ)
  - Nicht-optionale `FINAL`-Klausel erlaubt bzw. verbietet Subtypbildung
  - Optional: Bestimmen der Referenzgenerierung (OID-Erzeugung)
  - Optional: Objektverhalten in Methoden kodieren

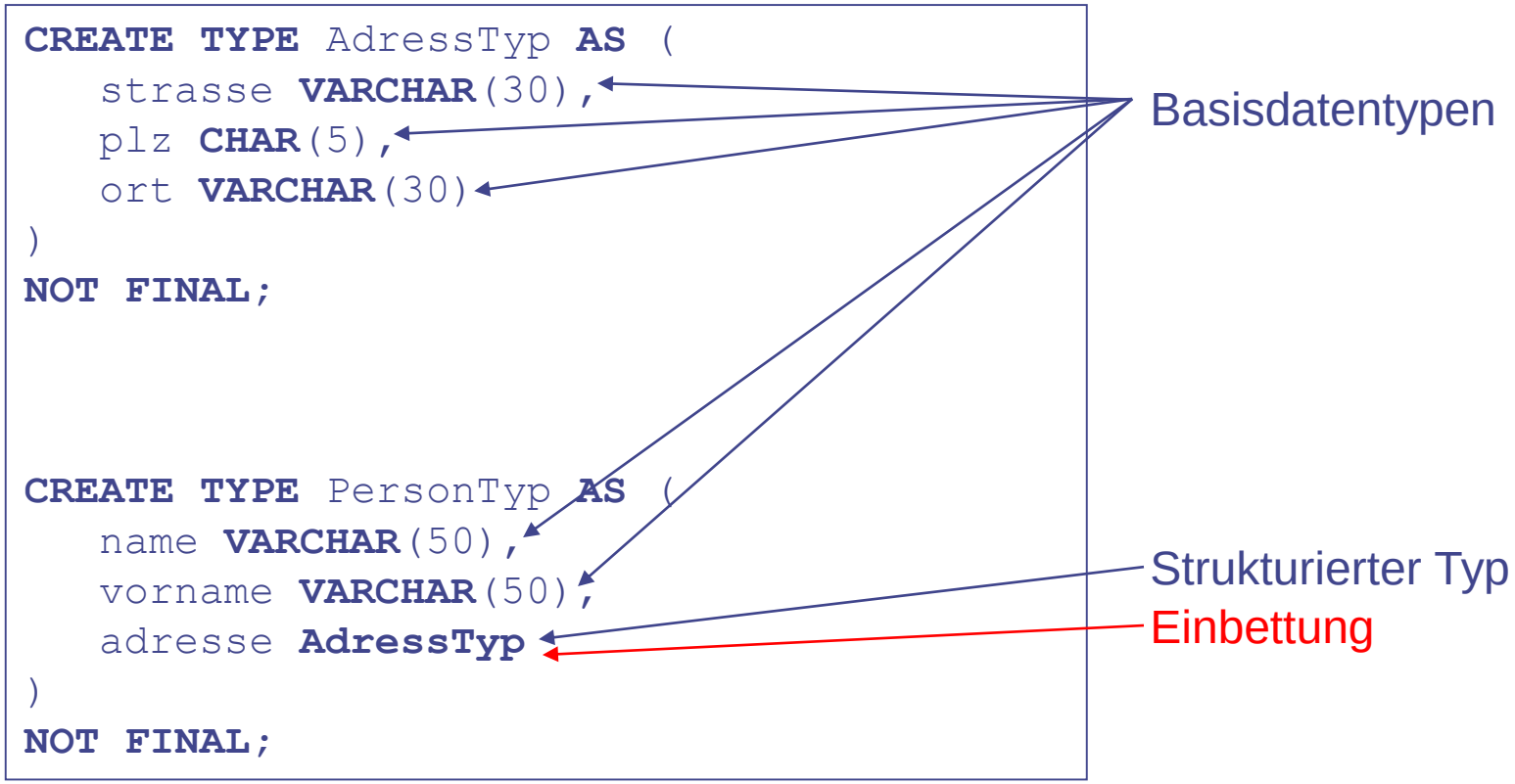
SQL:2008

Strukturierte Typen – Beispiel

```
CREATE TYPE AdressTyp AS (
    strasse VARCHAR(30),
    plz CHAR(5),
    ort VARCHAR(30)
)
NOT FINAL;



CREATE TYPE PersonTyp AS (
    name VARCHAR(50),
    vorname VARCHAR(50),
    adresse AdressTyp
)
NOT FINAL;
```



SQL:2008
Erzeugung von Instanzen

- Nutzung des Default-Konstruktors `Typname()`
- Definition + Nutzung eines überladenen Konstruktors

```
INSERT INTO Einwohner (name, vorname, adresse)
   VALUES ('Joy', 'Ann',
     AdressTyp('Langewiesener Str. 1', '98693','Ilmenau'));
```

SQL:2008
Vererbung

- Subtypdefinition mittels `UNDER`-Klausel
- Subtyp hat genau einen direkten Supertyp (keine Mehrfachvererbung)
- Supertyp muss selbst ein strukturierter Typ sein
- Subtyp erbt alle Attribute und Methoden des Supertyps
- Subtyp kann geerbte Methoden überschreiben und überladen

```
CREATE TYPE GrundstückTyp AS ( ... ) NOT FINAL;

CREATE TYPE BebautesGrundstückTyp
  UNDER GrundstückTyp AS ( ... ) NOT FINAL;
```

  - `NOT FINAL`: Subtyp-Bildung zulässig
  - `FINAL`: keine Subtyp-Bildung zulässig

SQL:2008
Methoden

- Methoden sind Funktionen, die zu einem strukturierten Typen gehören
  - Deklaration erfolgt innerhalb der Typdefinition
- Overloading, Overriding und Late Binding
  - Überladen von Methodennamen
    - mehrere gleichnamige Methoden mit unterschiedlichen Parametern
  - Dynamisches Binden von überschriebenen Methoden zur Laufzeit
    - Auswahl der Implementierung hängt vom strukturierten Typ ab
- Besonderheiten:
  - impliziter `SELF`-Parameter (vergleichbar `this` in C++, Java)
  - getrennte Spezifikation von Signatur und Implementierung
  - automatisch generierte `SET/GET`-Methoden für alle Attribute eines Typs

SQL:2008
Methoden – Beispiel

```
CREATE TYPE GrundstückTyp AS (
    ...
    ) NOT FINAL
    METHOD berechne_flaeche() RETURNS DECIMAL;


CREATE METHOD berechne_flaeche()
    RETURNS DECIMAL
    FOR GrundstückTyp
    BEGIN
      ...
    END;
```

Methodenimplementierung

SQL:2008
Strukturierte Typen für Tabellen

- Definition von Tabellen auf Basis strukturierter Typen
  ⇨ Objekttabellen
  - Attribute werden Spalten
  - zusätzliche Spalte für OID (benötigt für `REF`-Typen)

```
CREATE TABLE Einwohner OF PersonenTyp (
   REF IS OID SYSTEM GENERATED)
```

  - Referenzgenerierungsoptionen:
    - `REF IS OID USER GENERATED`
      OID wird vom Benutzer vorgegeben
    - `REF IS OID SYSTEM GENERATED`
      OID wird vom System generiert
    - `REF IS OID DERIVED (attributliste)`
      OID wird aus den Attributwerten abgeleitet (z.B. Primärschlüssel)

SQL:2008
REF-Typkonstruktor

- Typ für Referenzen auf Instanzen eines strukturierten Datentyps (Tupel einer Objekttabelle)
- Repräsentation von 1:1- oder n:1-Beziehungen

```
REF (strukturierter Typ)
   [SCOPE gültigkeitsbereich]
```

- Gültigkeitsbereich:
  - vorgegebene Tabelle
  - alle Tabellen
  - . . .

SQL:2008
REF – Beispiel

```
CREATE TYPE PersonTyp AS (
  name VARCHAR),
  vorname VARCHAR),
  gebdatum DATE);

CREATE TABLE Einwohner OF PersonTyp ( ...);

CREATE TABLE Grundstücke (
  nummer INT,
  lage AdressTyp,
  eigentümer REF(PersonTyp) SCOPE(Einwohner)
);
```

Scope-Klausel schränkt eine Referenz auf Instanzen in einer bestimmten Tabelle ein.

SQL:2008
REF-Operationen

- Belegung von Referenzen über Anfragen nach OID

```
UPDATE Grundstücke
SET eigentümer =
    ( SELECT OID
      FROM Einwohner
      WHERE name = 'Pren')
WHERE nummer = 42;
```

- Dereferenzierung

```
SELECT eigentümer->name
FROM Grundstücke;
```

- Vollständige Referenzauflösung: DEREF liefert den vollständigen Wert eines referenzierten Objektes

```
SELECT DEREF(eigentümer)
FROM Grundstücke;
```

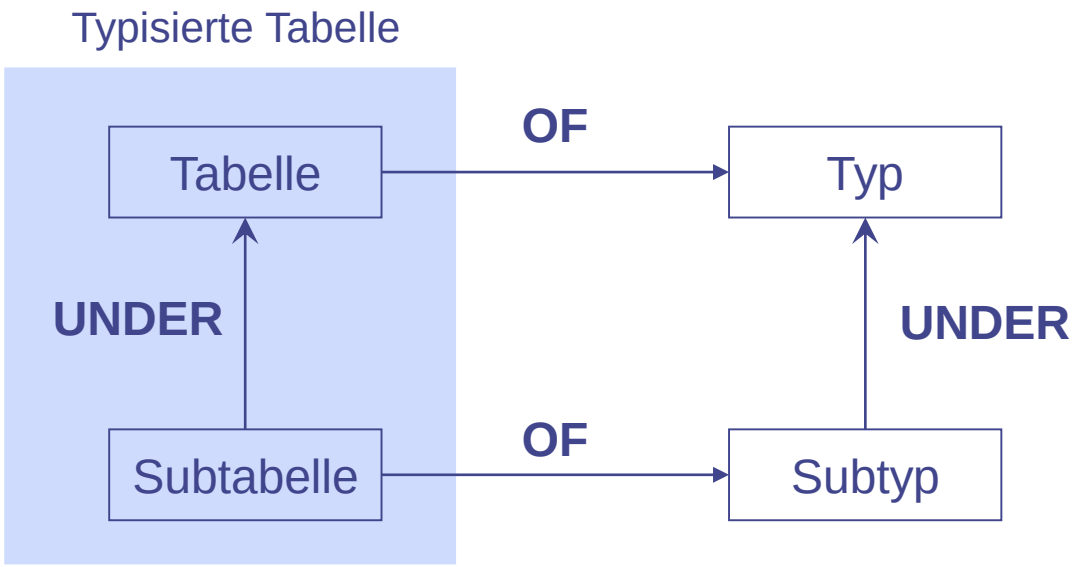
SQL:2008
Subtabellen

- Subtabellendefinition mittels `UNDER`-Klausel
  - Subtabelle hat genau eine direkte Supertabelle (keine direkte Mehrfachspezialisierung)
  - Typ der Subtabelle muss ein direkter Subtyp des Typs der Supertabelle sein
  - (Tiefe) Extension der Subtabelle muss immer eine Untermenge der (tiefen) Extension der Supertabelle sein
  - Instanzen der Subtabelle sind auch Mitglieder der zugehörigen Supertabellen
  - Subtabelle kann eigene Integritätsbedingungen definieren

```
CREATE TABLE subtabellenname OF typname
UNDER supertabellenname;
```

- Alle Zeilen der Subtabelle sind als Zeilen in der Supertabelle sichtbar (aber nur supertypspezifische Spalten)

SQL:2008
Zusammenhang zwischen Tabellen- und Typhierarchie



SQL:2008

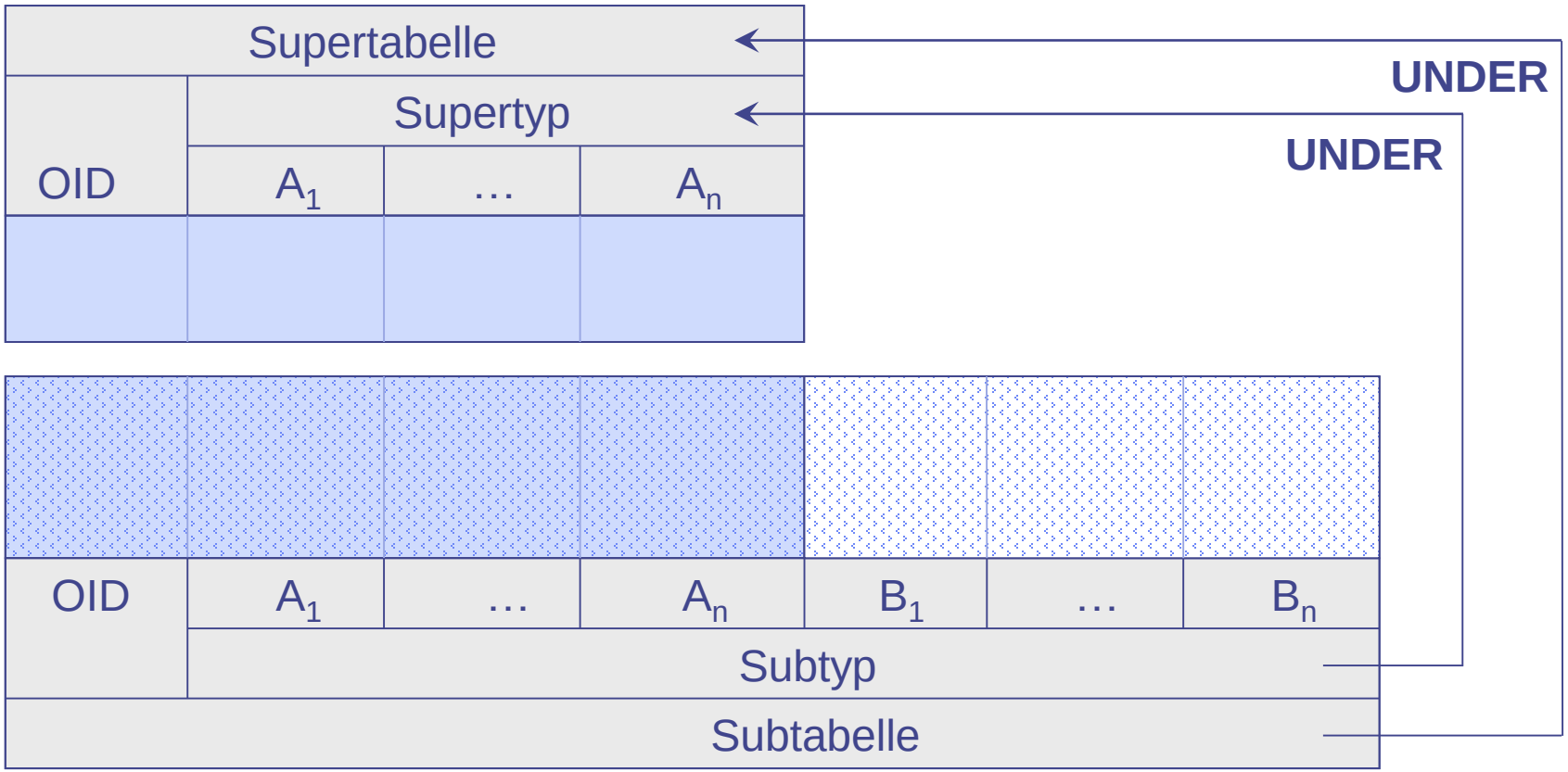
Tiefe und flache Extensionen 1/2

- Tiefe Extension: alle Tupel (Objekte) einer Tabelle einschließlich der Tupel der Subtabellen
- Flache Extension: ohne Tupel der Subtabellen
- Zugriff auf flache Extension: `ONLY(tabelle)`

```
SELECT * FROM ONLY(<supertabelle>);
```

SQL:2008

Tiefe und flache Extensionen 2/2



Flache Extension Supertabelle + Tiefe Extension Subtabelle = Tiefe Extension Supertabelle

SQL:2008
Typisierte Sichten

- Typisierte Sichten basieren analog zu typisierten Tabellen auf einem strukturierten Typen
- Unterschiede zu Tupelsichten
  - Zeilen über OID-Spalte referenzierbar; Methodenaufrufe möglich
  - typisierte Sicht als Subsicht einer anderen typisierten Sicht definierbar
- Arten von typisierten Sichten
  - objekterhaltende Sicht: basiert auf existierenden Objekten
  - objektgenerierende Sicht: erzeugt neue Objekte

SQL:2008
Typisierte Sichten: Definition

```
CREATE VIEW sichtname OF typname [(
   oid-generierung, ...
)]
AS sicht-anfrage
```

- Einschränkungen für die Sichtanfrage
  - darf nur eine Tabelle oder Sicht enthalten
  - ist diese Tabelle bzw. Sicht selbst typisiert, beschränkt sich der Zugriff auf die flache Extension (`ONLY`)
  - Verbundoperationen und Gruppierung sind nicht erlaubt

SQL:2008
Typisierte Sichten: Beispiel

```
CREATE TYPE GrundstückSichtTyp AS (
   nummer       INTEGER,
   lage         AdressTyp,
   eigentümer   REF(PersonTyp))
NOT FINAL
REF USING INTEGER;  -- Benutzerdefinierte Repräsentation

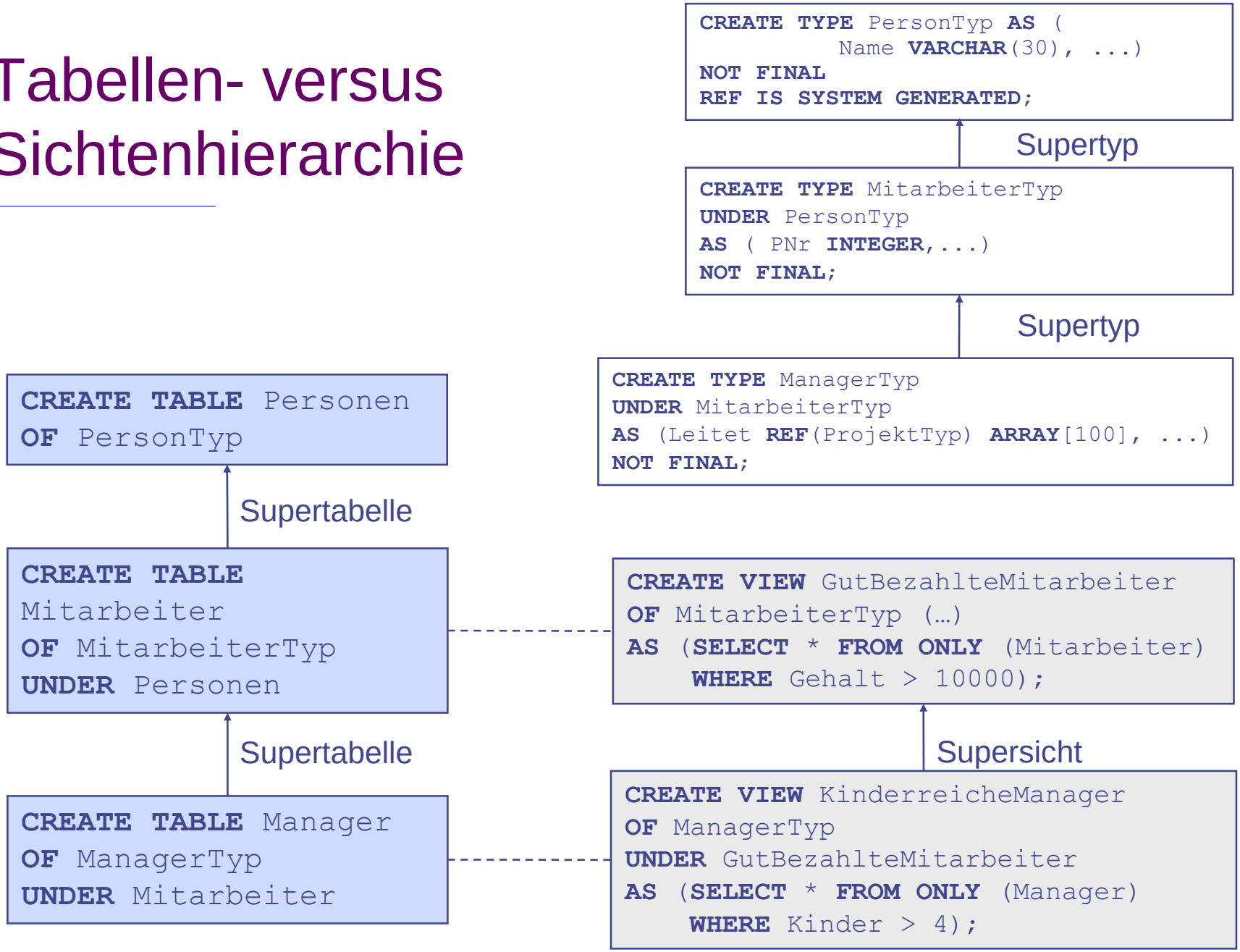
CREATE VIEW GroßeGrundstücke OF GrundstückSichtTyp (
   REF IS oid USER GENERATED,
   eigentümer WITH OPTIONS SCOPE Einwohner
)
AS SELECT GrundstückSichtTyp(nummer),  -- OID-Spalte
   nummer,
   AdressTyp(lage.strasse, lage.plz, lage.ort), -- Objekt
   PersonTyp(eigentümer->personID)
   FROM ONLY (Grundstücke)
   WHERE fläche > 500;
```

SQL:2008
Subsichten

- Subsicht hat genau eine direkte Supersicht (keine Mehrfachspezialisierung)
- Typ der Subsicht muss ein direkter Subtyp des Typs der Supersicht sein
- Extension der Subsicht ist immer eine Untermenge der Extension der Supersicht
  - Genauer: Subsicht erweitert die Extension der Supersicht

```
CREATE VIEW sichtname OF typname
UNDER supersichtname
AS sicht-anfrage
```

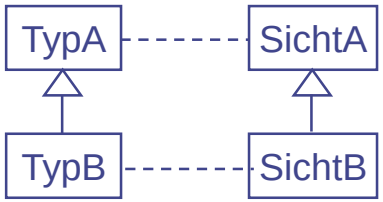
Tabellen- versus Sichtenhierarchie



Anhang II

Sichthierarchien

Sichten
Sichthierarchien



- Objektsichten lassen sich in Sichthierarchien organisieren

```
CREATE [OR REPLACE][FORCE] VIEW <SichtB> OF <TypB>
  UNDER <SichtA>
  AS <SFW-Block>
  [WITH {READ ONLY | CHECK OPTION}];
```

- Obersicht (`SichtA`) muss zu direktem Obertyp (`TypA`) des aktuellen Typs (`TypB`) definiert sein
- Über die Obersicht sind automatisch auch die Objekte aller Subsichten zugreifbar (Extent-Inklusion)
- `ONLY`-Klausel, um nur auf die Objekte einer bestimmten Sicht, ohne Objekte der Subsichten, zuzugreifen

```
SELECT * FROM ONLY (SichtA);
```

Sichten

Sichthierarchien

Obersicht (Wurzelsicht)

```
CREATE OR REPLACE VIEW LinzKunde OF Kunden_Typ
AS SELECT *
  FROM Kunde k
  WHERE VALUE(k) IS OF (ONLY Kunden_Typ)AND
    k.Adresse.Ort = 'Linz';
```

Untersicht (erweitert Obersicht LinzKunde)

```
CREATE OR REPLACE VIEW LinzVIPKunde OF VIPKunden_Typ UNDER LinzKunde
AS SELECT *
  FROM VIPKunde k
  WHERE VALUE(k) IS OF (ONLY VIPKunden_Typ) AND
    k.Adresse.Ort = 'Linz';
```